WEEK 3 · LECTURE

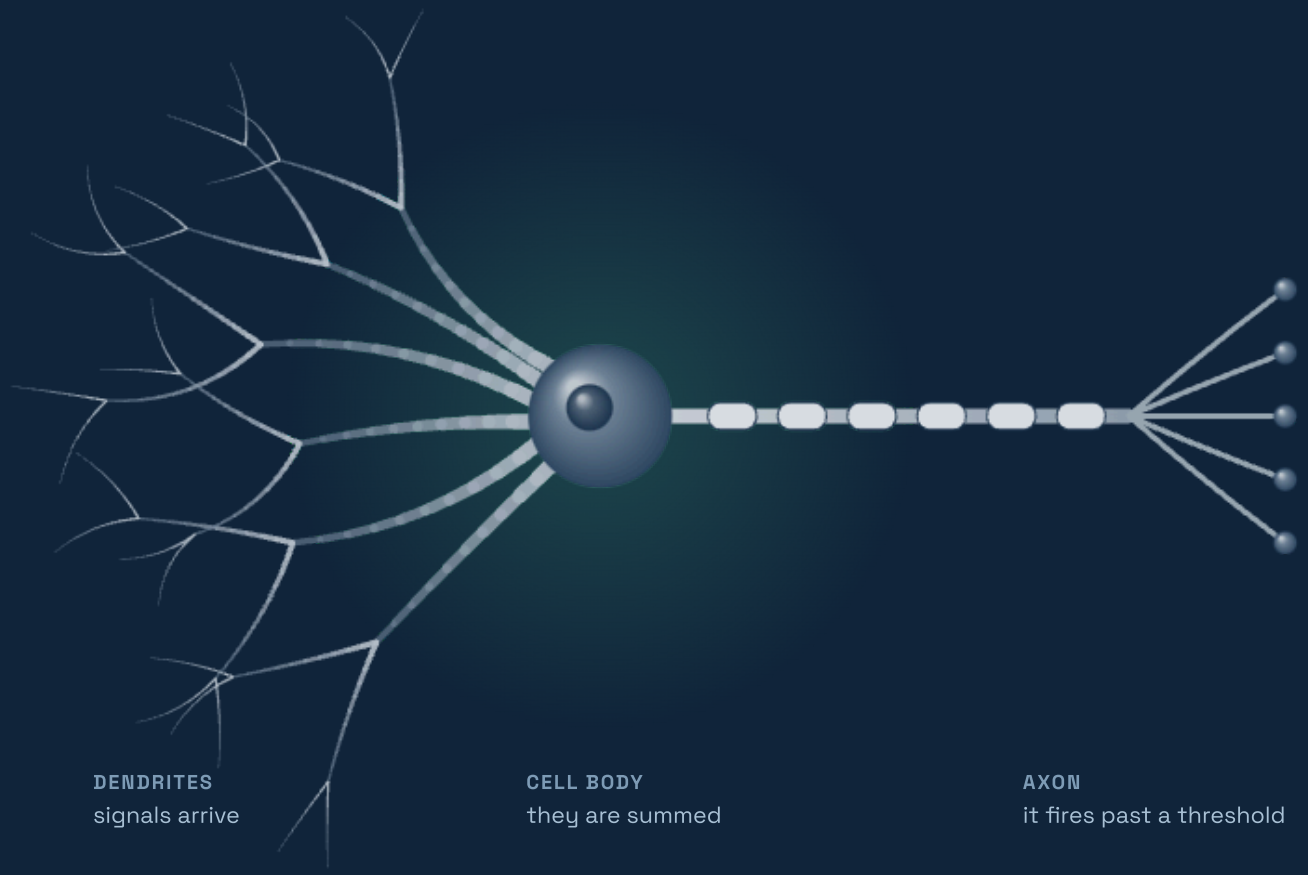
Learning from labels: classifiers, neural networks & embeddings

Dr. Philipp K. Masur
Computational Analysis of Digital Communication · Vrije Universiteit Amsterdam

VU VRIJE UNIVERSITEIT AMSTERDAM

A machine that learns by doing

A NEURON



DENDRITES
signals arrive

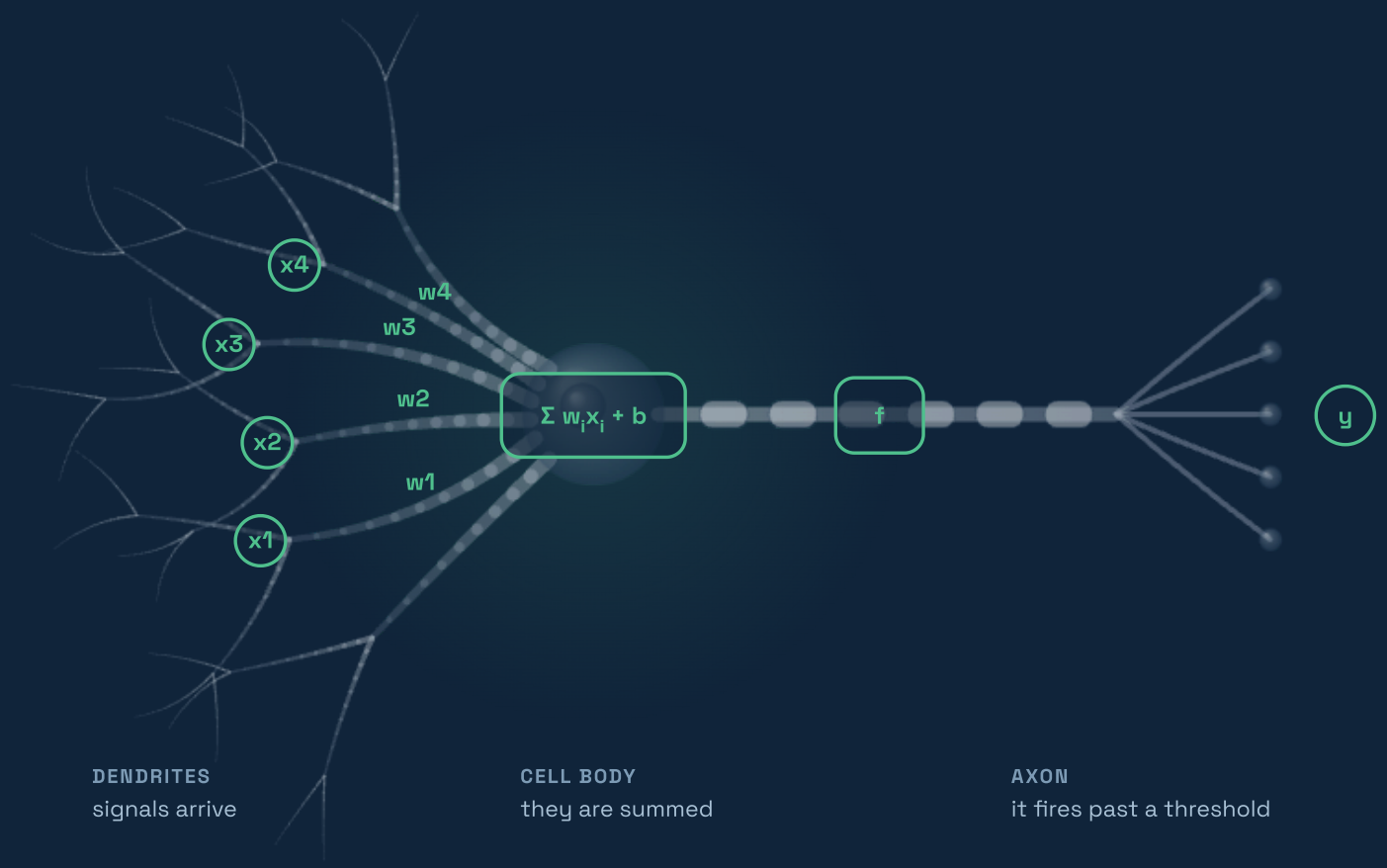
CELL BODY
they are summed

AXON
it fires past a threshold

< 1 / 8 >

A machine that learns by doing

THE UNIT, 1943



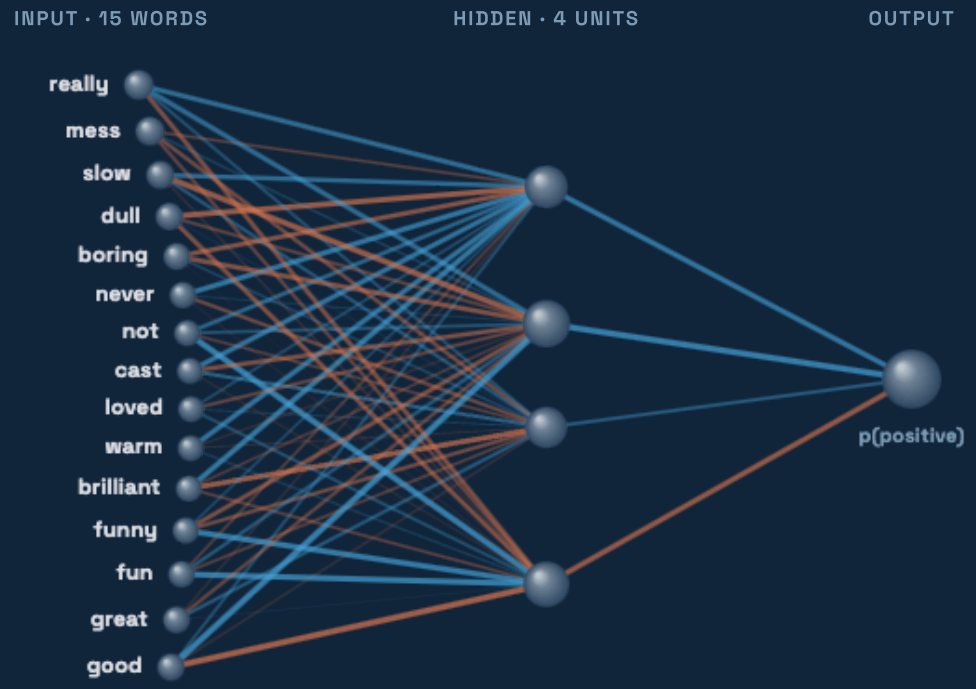
DENDRITES
signals arrive

CELL BODY
they are summed

AXON
it fires past a threshold

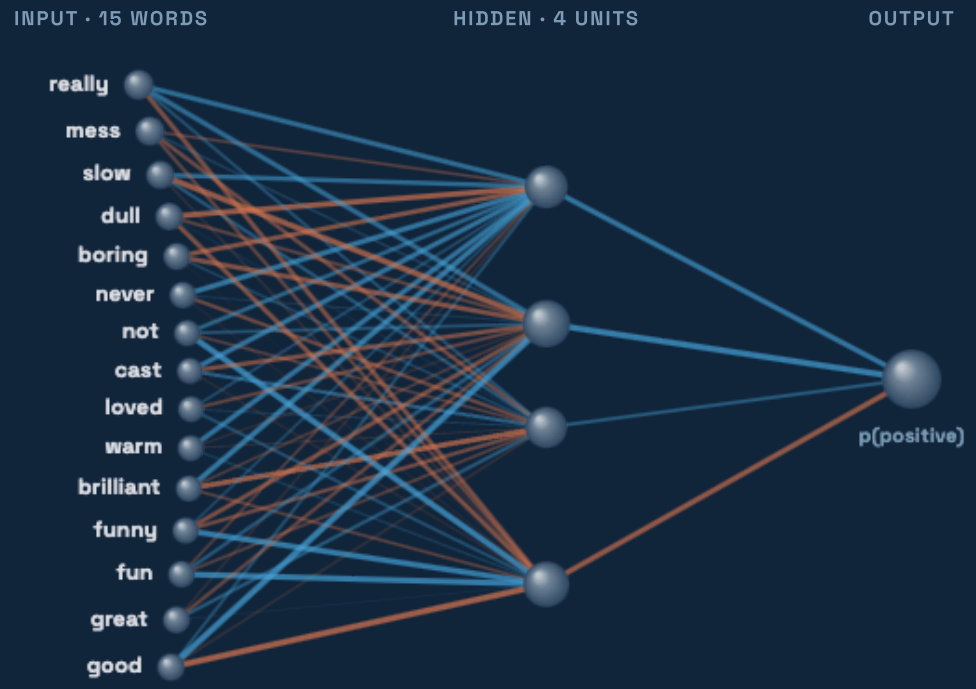
< 2 / 8 >

A machine that learns by doing



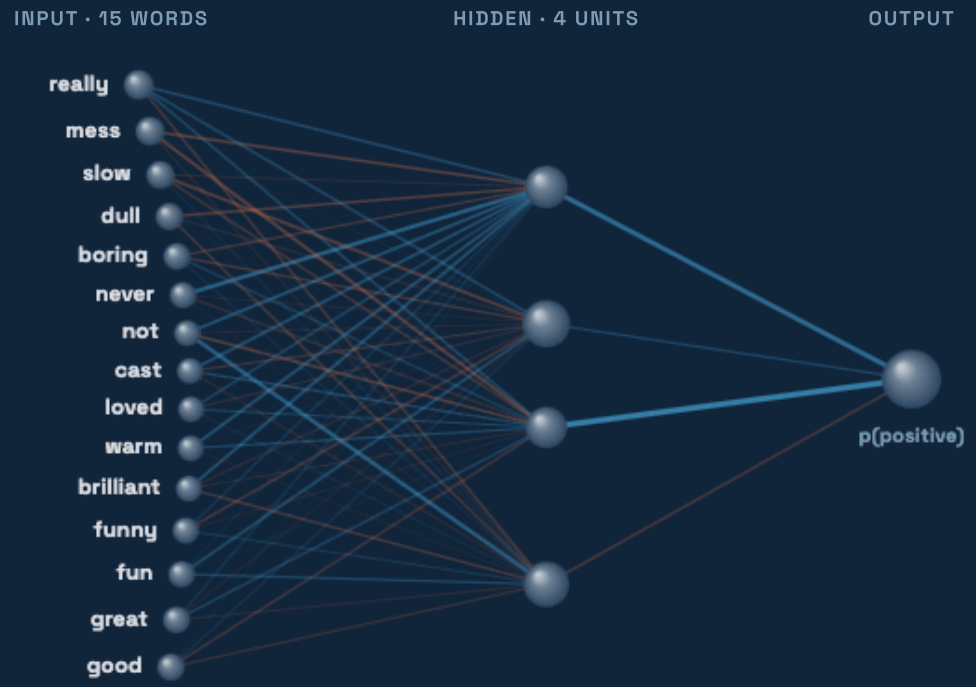
TWENTY REVIEWS	HUMAN
Good, really good.	+
Not good at all.	−
A great cast.	+
Not funny.	−
Great fun.	+
Dull and slow.	−
Brilliant and funny.	+
Boring.	−
Funny and warm.	+
A boring, slow mess.	−
Loved every minute.	+
Never good.	−
Not boring at all.	+
Not great, not funny.	−
Never dull.	+
Slow and dull.	−
Good fun, great cast.	+
A mess.	−
Warm and brilliant.	+
Not brilliant.	−

A machine that learns by doing



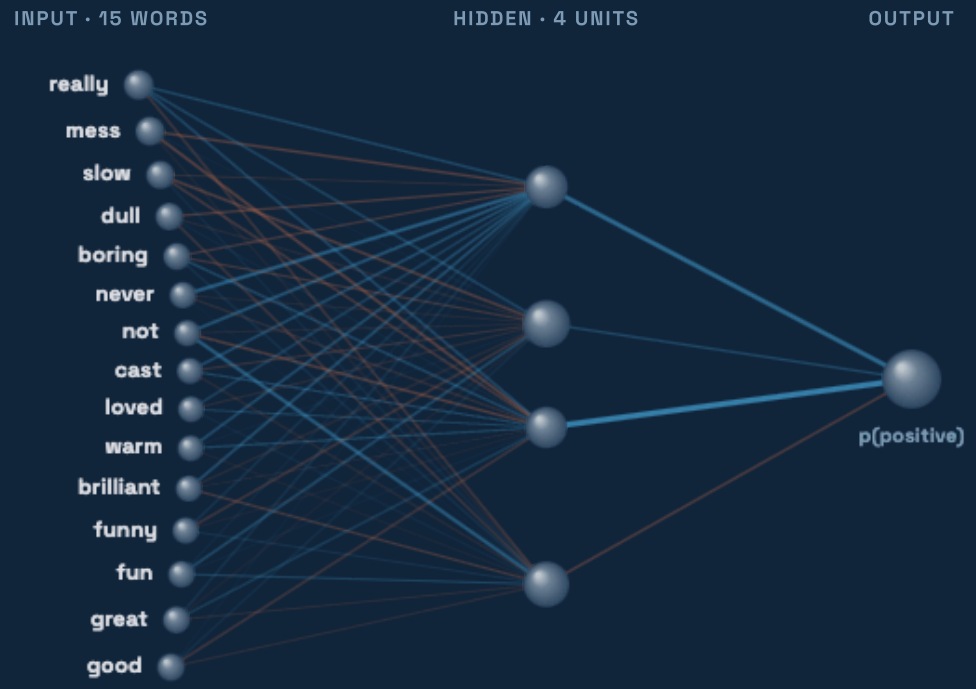
TWENTY REVIEWS	HUMAN	NETWORK	
Good, really good.	+	+	✓
Not good at all.	−	+	✗
A great cast.	+	+	✓
Not funny.	−	−	✓
Great fun.	+	−	✗
Dull and slow.	−	+	✗
Brilliant and funny.	+	−	✗
Boring.	−	−	✓
Funny and warm.	+	−	✗
A boring, slow mess.	−	−	✓
Loved every minute.	+	−	✗
Never good.	−	+	✗
Not boring at all.	+	−	✗
Not great, not funny.	−	−	✓
Never dull.	+	−	✗
Slow and dull.	−	+	✗
Good fun, great cast.	+	+	✓
A mess.	−	−	✓
Warm and brilliant.	+	+	✓
Not brilliant.	−	−	✓

A machine that learns by doing



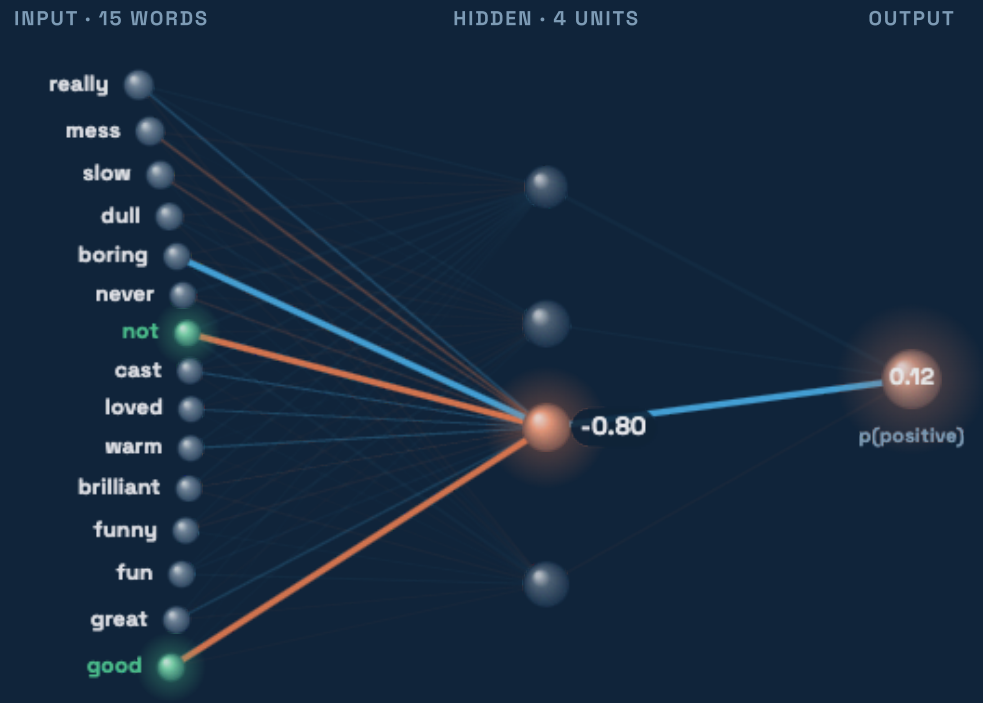


A machine that learns by doing





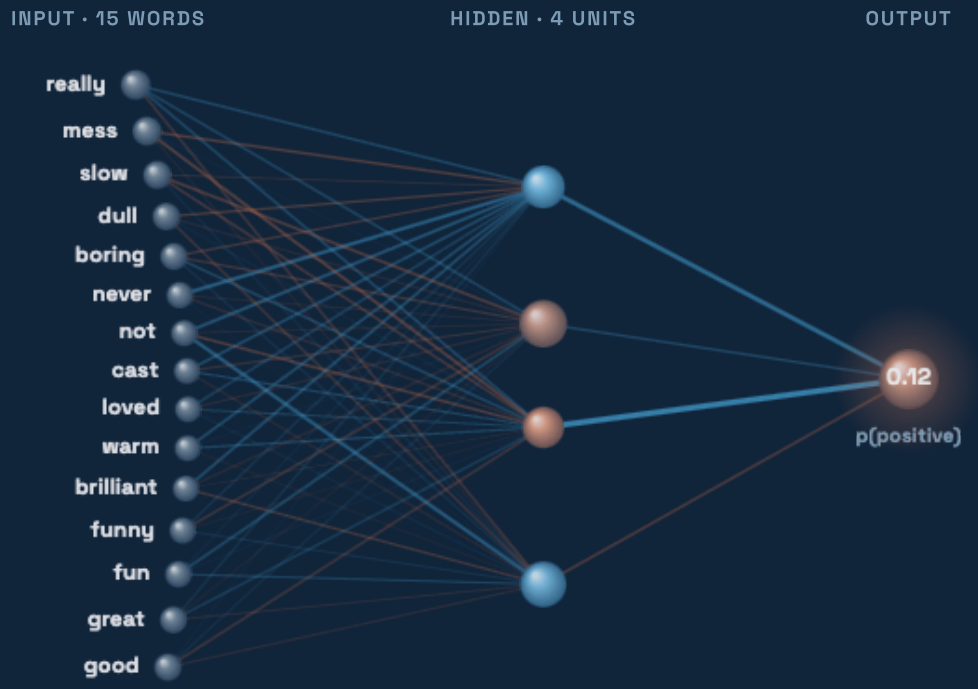
A machine that learns by doing



This unit turns **red** for “Not good at all.” and **blue** for “Not boring at all.”



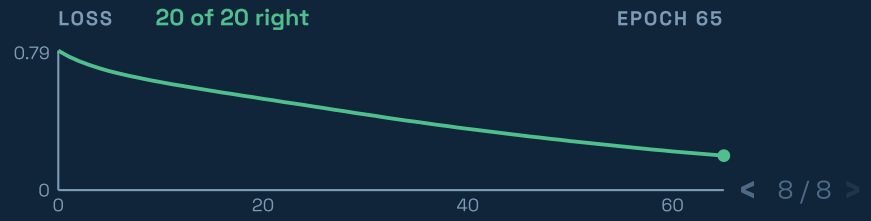
A machine that learns by doing



20 of 20.

It has seen these twenty reviews 65 times. **Would it get the next twenty?**

TWENTY REVIEWS	HUMAN NETWORK		
Good, really good.	+	+	✓
Not good at all.	−	−	✓
A great cast.	+	+	✓
Not funny.	−	−	✓
Great fun.	+	+	✓
Dull and slow.	−	−	✓
Brilliant and funny.	+	+	✓
Boring.	−	−	✓
Funny and warm.	+	+	✓
A boring, slow mess.	−	−	✓
Loved every minute.	+	+	✓
Never good.	−	−	✓
Not boring at all.	+	+	✓
Not great, not funny.	−	−	✓
Never dull.	+	+	✓
Slow and dull.	−	−	✓
Good fun, great cast.	+	+	✓
A mess.	−	−	✓
Warm and brilliant.	+	+	✓
Not brilliant.	−	−	✓



> The Navy revealed the embryo of an electronic computer today that it expects will be able to walk, talk, see, write, reproduce itself and be conscious of its existence. [...] Later perceptrons will be able to recognize people and call out their names and instantly translate speech in one language to speech and writing in another language, it was predicted.
>
> *The New York Times*, 8 July 1958, on Frank Rosenblatt's perceptron

Sixty-eight years later, the last sentence is true and the first is not. This lecture is about what the machine in between actually does.

New Navy device learns by doing: Psychologist shows embryo of computer designed to read and grow wiser. (1958, July 8). *The New York Times*, p. 25.

Last week ended here

- A dictionary is a vector of weights over the columns of the document–term matrix (+1, −1, 0), **declared by you** before the machine sees a text
- Against a hand-coded gold standard, no dictionary reached α = .35; the classifiers in the same paper reached .39 to .50, the humans .75 and up
- The list was not wrong about the words. It was wrong about the texts, and *not good* is the reason
- This week the weights are **learned from labelled texts**, and the result is scored against human coding exactly as the word list was

> **Same matrix, same validation. What changes is who writes the weights.**

This lecture

I

Learning from labels

Inductive instead of deductive; the pipeline; what a model is for

II

Train, test, overfit

Why a model that fits perfectly is the one to distrust

III

A classifier, end to end

One tidymodels workflow from split to F1, then swap the algorithm

IV

Neural networks

The unit, the hidden layer, and how the weights are found

V

Word embeddings

Words as vectors; a representation the model did not have to learn

VI

Weeks 1 to 3 in one table

Where every method so far lands against the same gold standard

I · Learning from labels

Deductive and inductive

- A dictionary is **deductive**: the rule comes first (*happy* means positive), and the texts are read through it
- It is also deterministic: the same words produce the same score, however they are arranged
- A human coder does something else: reads the whole sentence, weighs the words against each other, decides in context
- **Inductive** approaches start from the coder's decisions and let the machine work out which features of a text predict them
- The promise: the scalability of the dictionary with the validity of the coder (never quite kept, as Part VI shows)

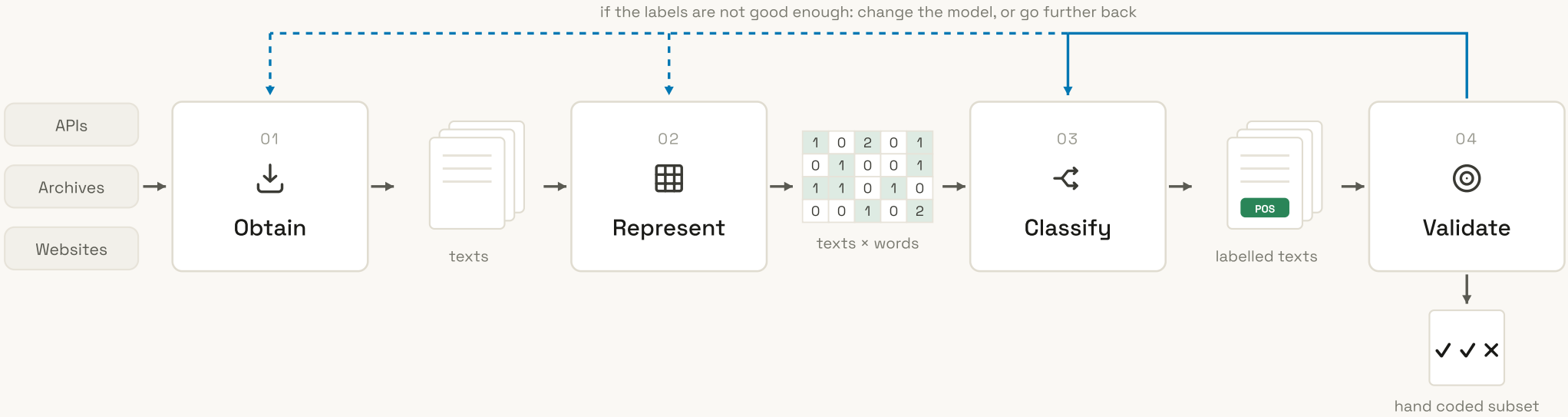
> **Deductive: the researcher supplies the rules. Inductive: the researcher supplies examples, and the rules are inferred.**

Four ways of assigning meaning

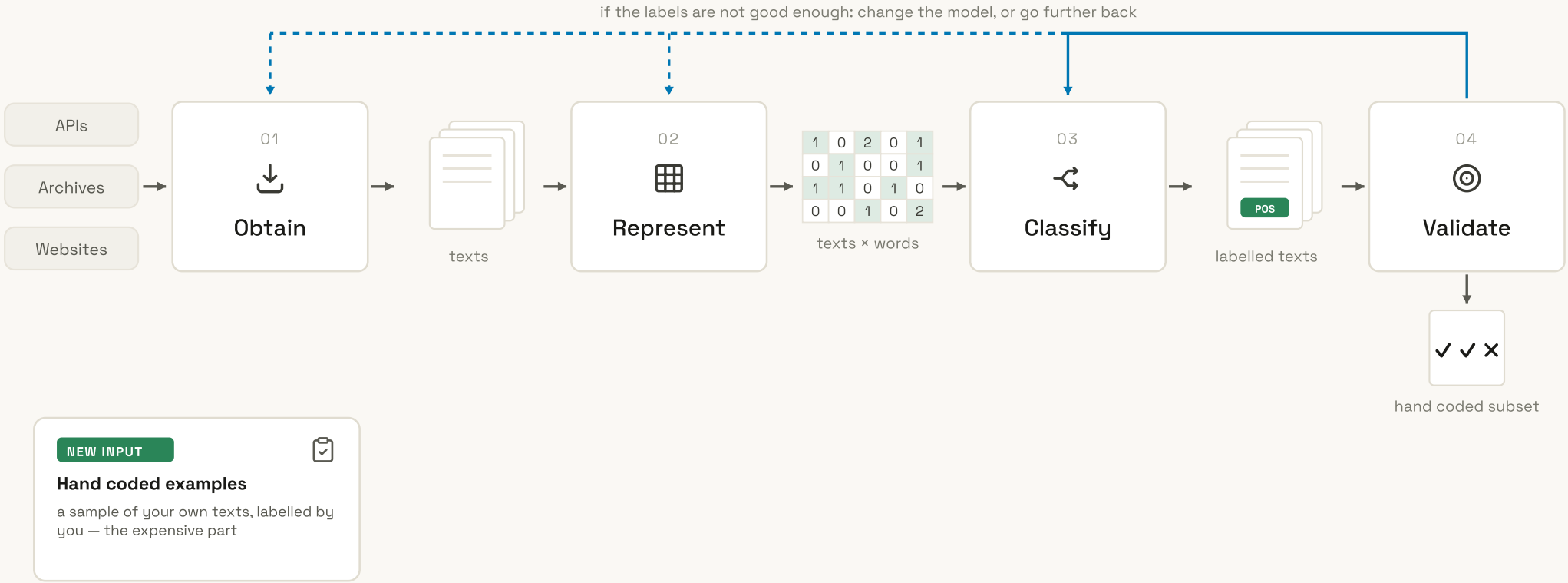
	RULE BASED	UNSUPERVISED	SUPERVISED	PRE-TRAINED
Logic	*if word X occurs, the text means Y*	*these words form a pattern, which I think means X*	*X resembles texts labelled negative, so X is negative*	*the model treats X as it treats negative texts*
Meaning assigned	by the researcher, in advance	by the researcher, afterwards	learned from the coded training set	by the model, from its own training
Example	dictionaries	topic models	Naive Bayes, SVM, neural networks	large language models
Week	2	—	**3**	4

> **Today is the third column. The fourth is next week, and it is the third with the human coding moved somewhere you cannot see.**

The pipeline with a classifier

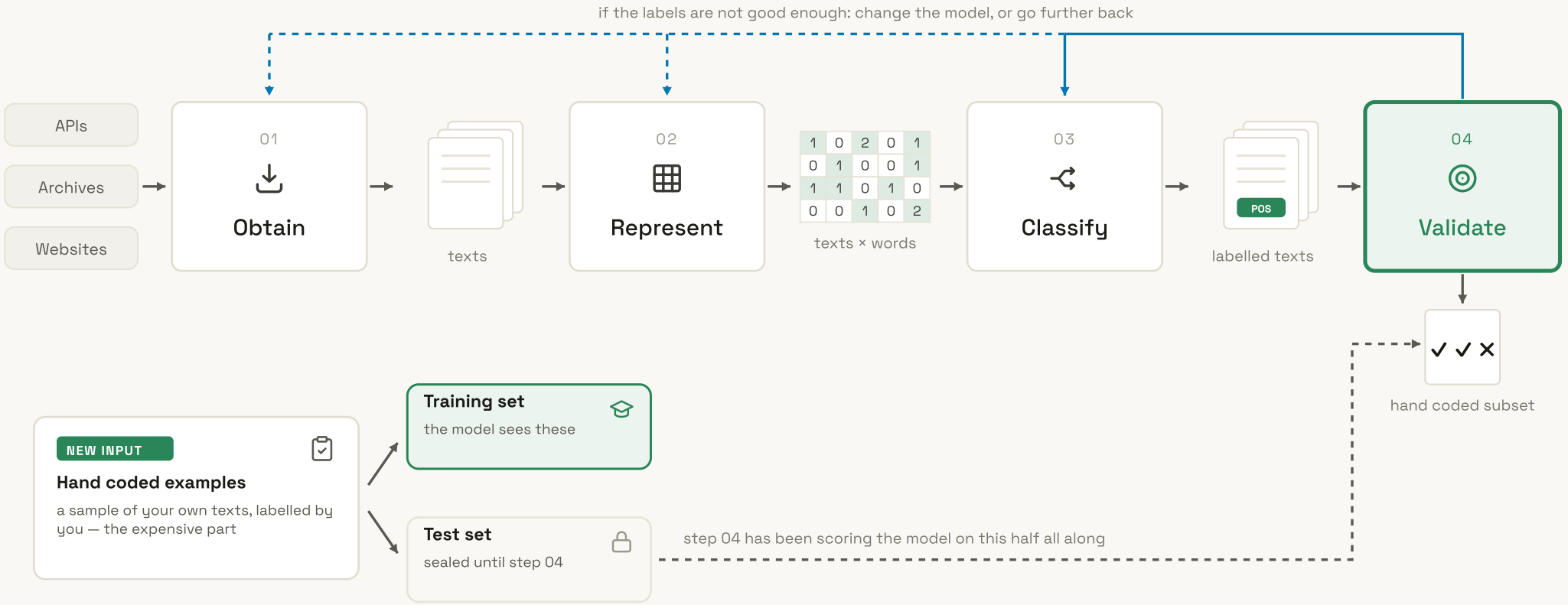


The pipeline with a classifier



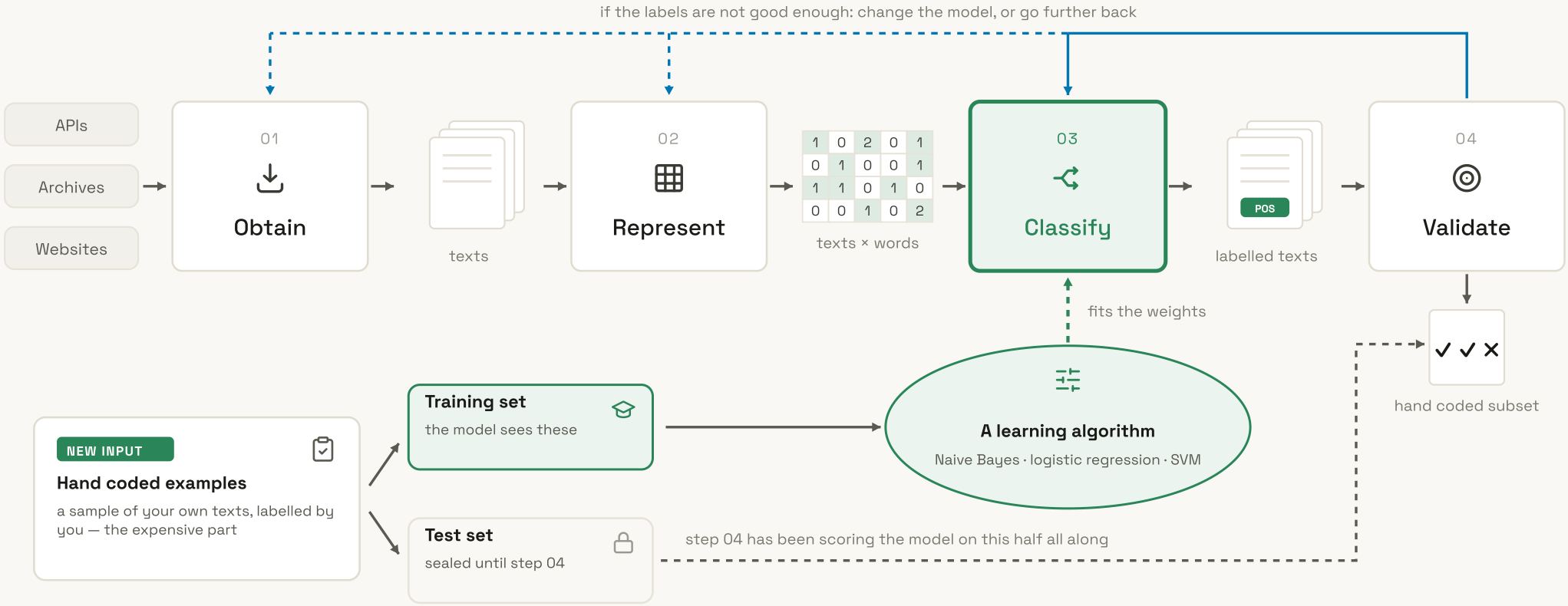
2 / 5

The pipeline with a classifier

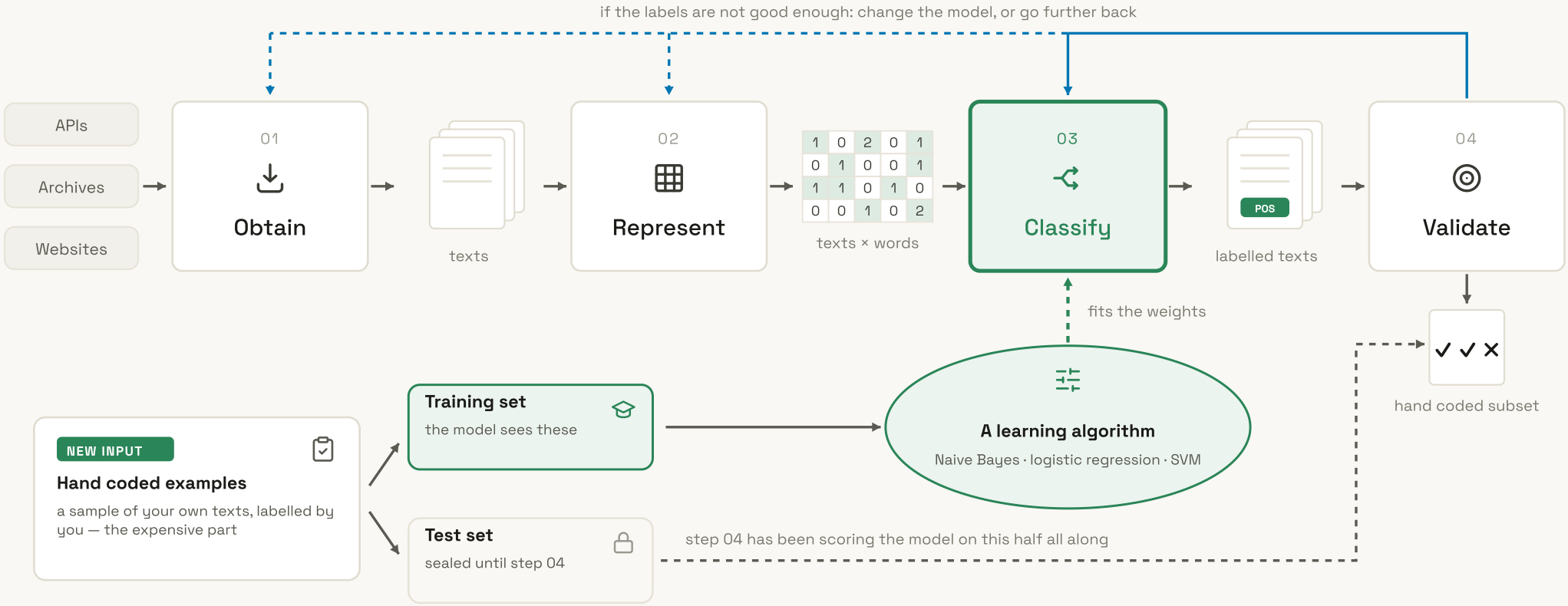


3 / 5

The pipeline with a classifier



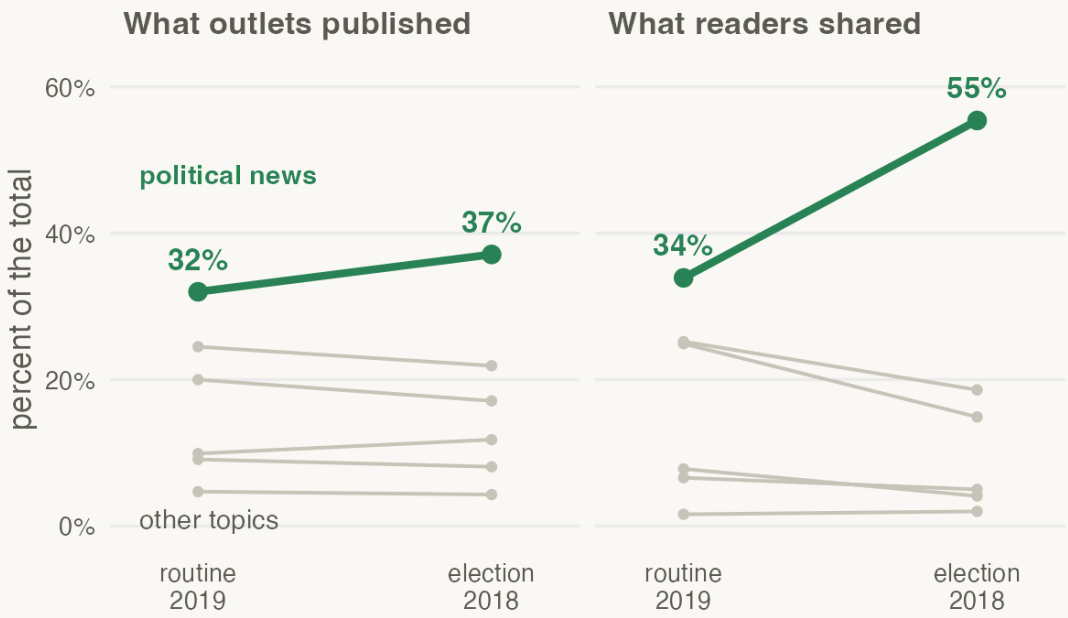
The pipeline with a classifier



5 / 5

A worked case

- de León, Vermeer and Trilling (2023): how do elections change what Mexican news outlets publish and what readers share on Facebook?
- 83,054 articles, far too many to code by hand. 2,000 were coded into six topics, and a support vector machine was trained on 80% of them and tested on the rest
- With precision and recall above .75 on every topic, it labelled the full sample



> **Two thousand coded, eighty-one thousand classified. The ratio is the method's whole appeal, and the test on held-out articles is what makes it defensible.**

de León, E., Vermeer, S., & Trilling, D. (2023). Electoral news sharing: A study of changes in news coverage and Facebook sharing behaviour during the 2018 Mexican elections. *Information, Communication & Society, 26*(6), 1193–1209. https://doi.org/10.1080/1369118X.2021.1994629 Figure drawn from their Table 1.

Statistical modelling: learn *f*

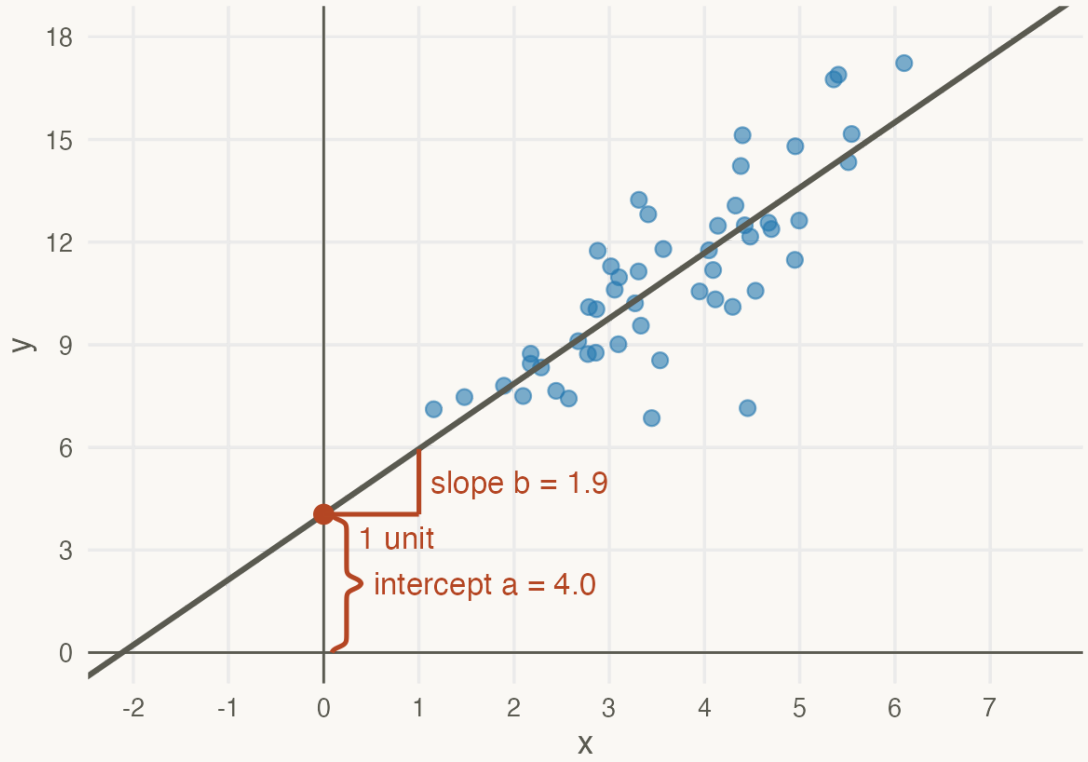
```r
set.seed(311)
data <- tibble(x = rnorm(50, 3.5, 1.2),
               y = 4 + 2 * x + rnorm(50, 0, 2))

fit <- lm(y ~ x, data = data)
coef(fit)
```

OUTPUT

```
(Intercept)           x
   4.042616    1.909887
```

- Fifty observations, one predictor, one outcome: the model is a line, $y = a + b \cdot x$
- Fitting finds the a and b that make the line pass closest to the points
- The regression reading: at $x = 0$ the line is at 4.0, and one unit more x gives 1.9 units more y



Prediction: use *f* on an *x* you never saw

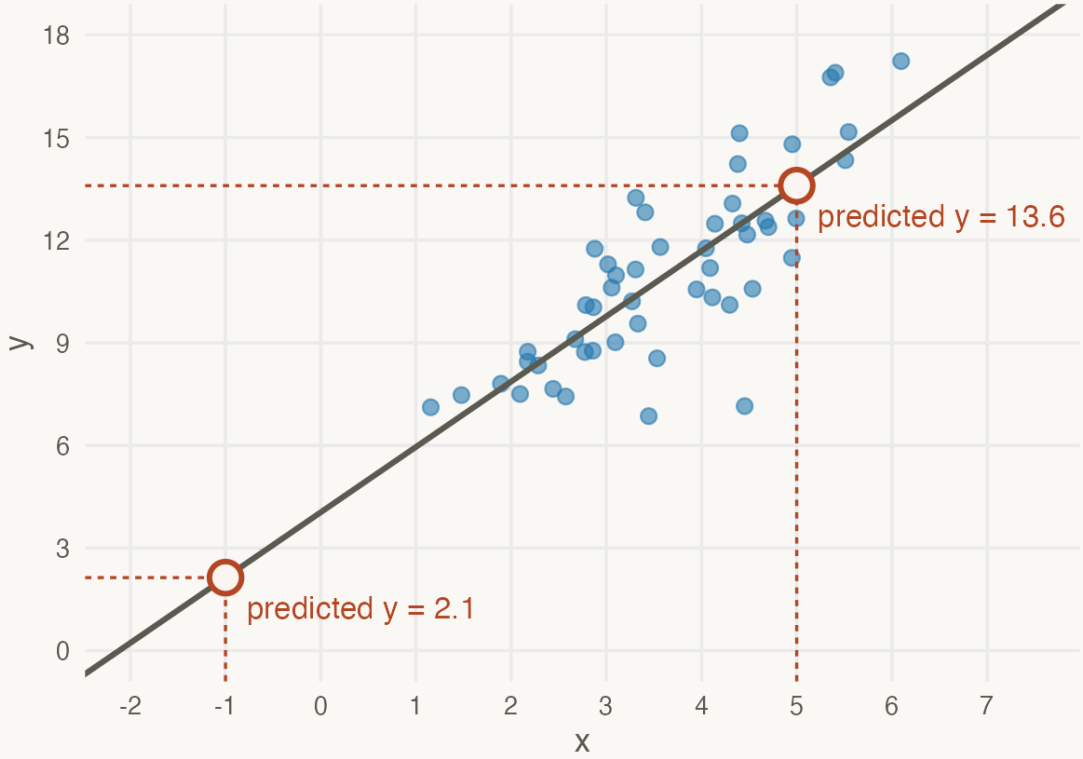
```
new <- tibble(x = c(-1, 5))

predict(fit, newdata = new)
```

OUTPUT

```
       1        2
2.132729 13.592050
```

- No observation has $x = -1$ or $x = 5$, but a y is returned for each anyway
- **Machine learning** is this second use, taken seriously: fit on what you have, predict what you do not
- The line stays the same; you ask it a different question



Two vocabularies for one thing

MACHINE LEARNING SAYS	STATISTICS SAYS
feature	independent variable, predictor
label	dependent variable, outcome
labelled data set	a data set with both
to train a model	to estimate, to fit
classifier	a model for a nominal outcome
to annotate	to code, by hand
training and test set	the sample, and the data you never have

> **The last row is the real difference: statistics rarely holds data back to check its own predictions. Machine learning always does.**

van Atteveldt, W., Trilling, D., & Arcíla Calderón, C. (2022). *Computational analysis of communication*. Wiley. Chapter 8.

Two goals, and what each gives up

EXPLAINING

- Explaining is what **statistics** is about
- Does *X* relate to *Y*, in the population?
- Few predictors, so each coefficient can be read and defended
- The parameters are the result

PREDICTING

- Predicting is what **text classification** and **machine learning** are about
- How well can *Y* be guessed from what we have?
- Thousands of predictors, most of them collinear; nobody reports them
- The predictions are the result; the parameters are a means

> **In this course the classifier's output becomes a variable in a study. Its job is to predict; the explaining happens afterwards, with the variable it produced.**

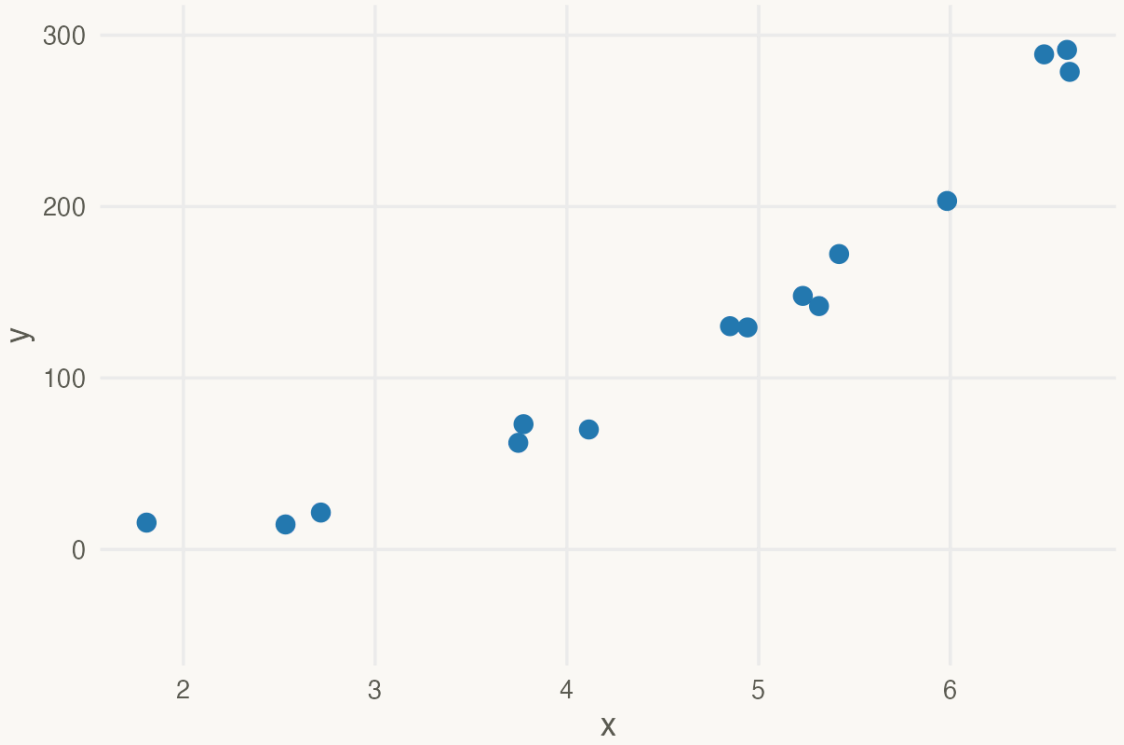
II · Train, test, overfit

Fifteen points

```r
set.seed(42)
sim <- tibble(x = runif(15, 1, 7),
              y = x^3 + rnorm(15, 0, 10))

ggplot(sim, aes(x, y)) +
  geom_point()
```

- Simulated, so the truth is known: *y* is *x* cubed plus noise
- The next three slides fit three models to these fifteen points and ask which one to trust
- The Playground's *How much freedom is too much* is this simulation with a slider

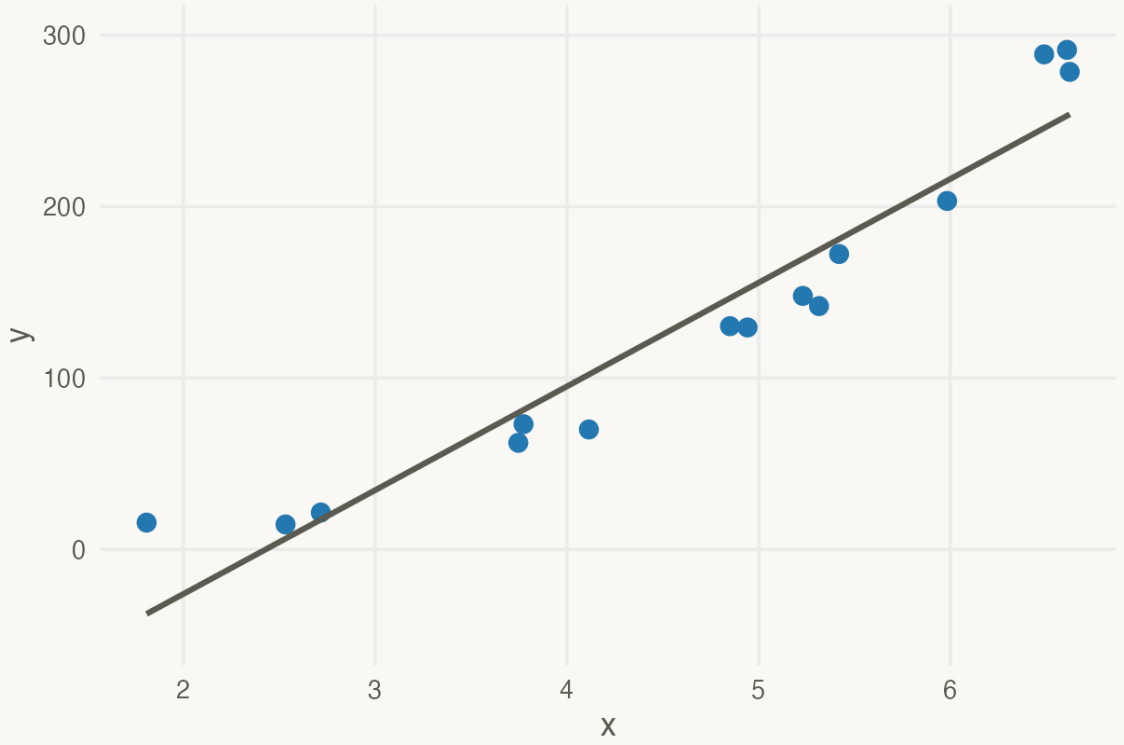


PLAYGROUND How much freedom is too much

A line: too little freedom

```
ggplot(sim, aes(x, y)) +
  geom_point() +
  geom_smooth(method = "lm",
              formula = y ~ x)
```

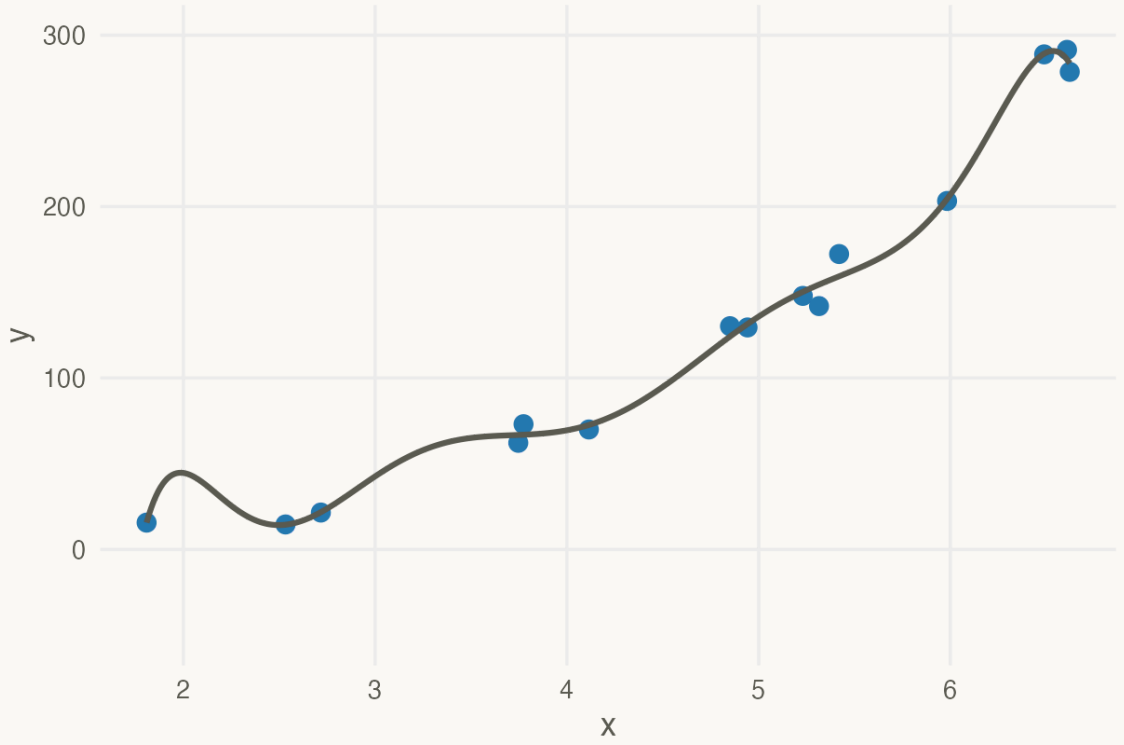
- Two parameters. The line misses the curve systematically: too high on the left, too low on the right
- **Underfitting**: the model is too simple for the pattern in the data
- The errors are large on the training data already, and would be on new data too



A ninth-degree polynomial: too much

```
ggplot(sim, aes(x, y)) +
  geom_point() +
  geom_smooth(method = "lm",
              formula = y ~ poly(x, 9))
```

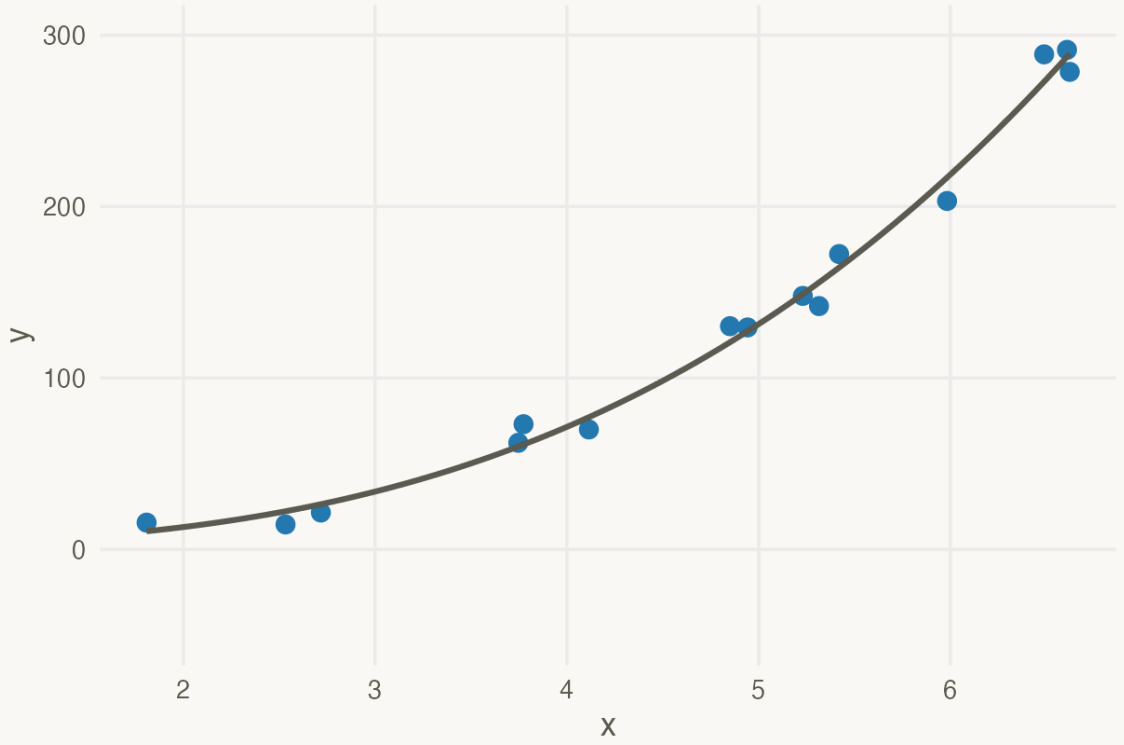
- Ten parameters for fifteen points. The curve visits every observation, noise included
- **Overfitting**: the model has learned the sample, not the process
- Its training error is nearly zero. Ask what it would predict for $x = 6.5$



A cubic: about right

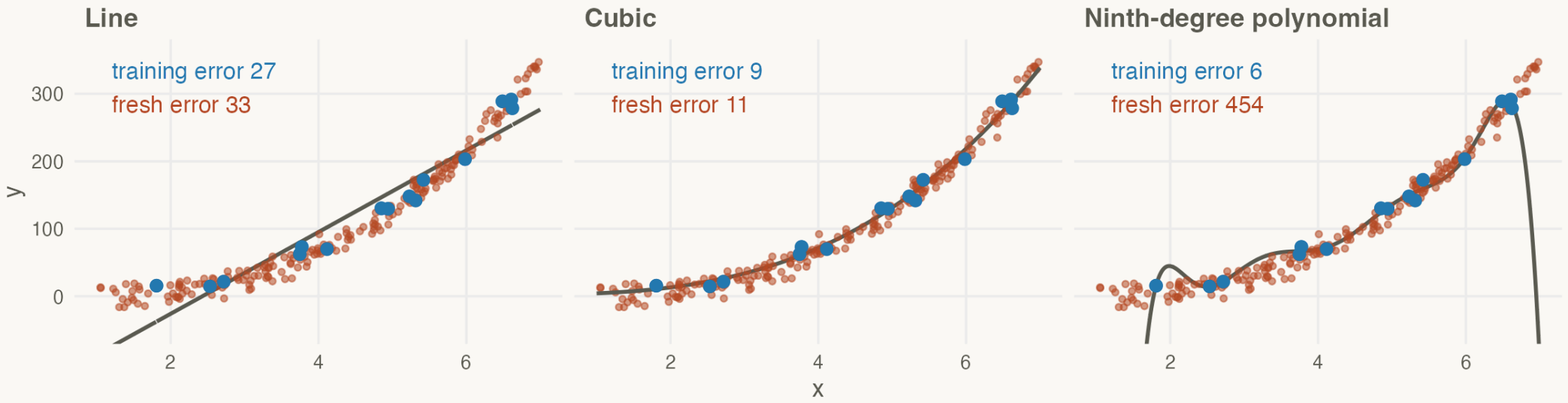
```
ggplot(sim, aes(x, y)) +
  geom_point() +
  geom_smooth(method = "lm",
              formula = y ~ poly(x, 3))
```

- Four parameters, and the shape of the process that made the data
- Misses individual points, follows the curve: the errors that remain are the noise
- In a real problem nobody tells you the degree. Something has to stand in for knowing the truth



What stands in for the truth: data the model never saw

- Let the three models predict 200 fresh points from the same process



> You will never have the fresh two hundred. So you make them: hold part of your labelled data back before you fit anything.

Hold part back

5,250 LABELLED ABSTRACTS · ONE TILE = 50



computer science · 1,750 physics · 1,750 statistics · 1,750

As the file came: sorted by discipline. Cut here and the test set would be **all statistics**.

1 / 5

Hold part back

5,250 LABELLED ABSTRACTS · ONE TILE = 50



set.seed(42) shuffled once, reproducibly

Hold part back

5,250 LABELLED ABSTRACTS · ONE TILE = 50

TRAIN · 4,200 · 80%

TEST · 1,050



set.seed(42) shuffled once, reproducibly

Four abstracts in five train the model. The fifth is **sealed**: nothing below the cut looks at it until the very end.

3 / 5

Hold part back

5,250 LABELLED ABSTRACTS · ONE TILE = 50

TRAIN · 4,200 · 80%

TEST · 1,050



set.seed(42) shuffled once, reproducibly

TRAIN, BY CLASS

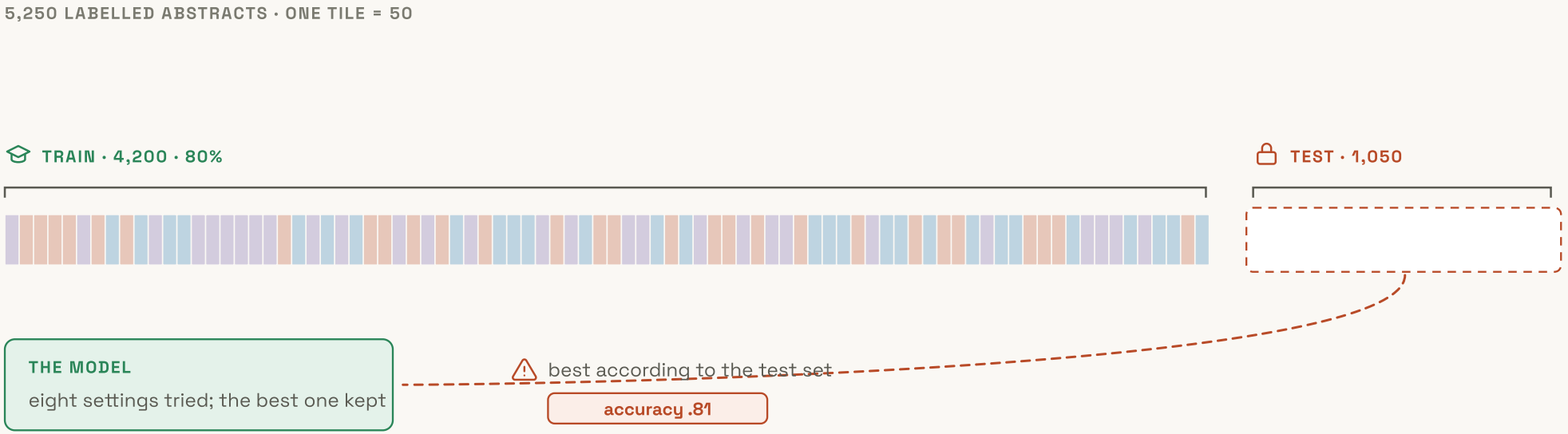
33% · 33% · 33%

TEST, BY CLASS

33% · 33% · 33%

strata = label: each class keeps its share on both sides, so the test set is a fair sample of the whole.

Hold part back



Choosing a setting because it scored best on the test set is **leakage**: the test set has quietly become training data, and .81 is no longer an estimate of anything. The score you report has to come from data no decision ever touched.

Other designs

HOLD-OUT · THE SPLIT YOU JUST SAW



Enough when there is nothing to choose.

But every model has settings, and choosing them needs a score too. **Not the test score.**

< 1 / 8 >

Other designs

TRAIN · VALIDATION · TEST



A third part, cut from the training data, takes every decision.

The price: 1,050 coded abstracts that never train anything.

Other designs

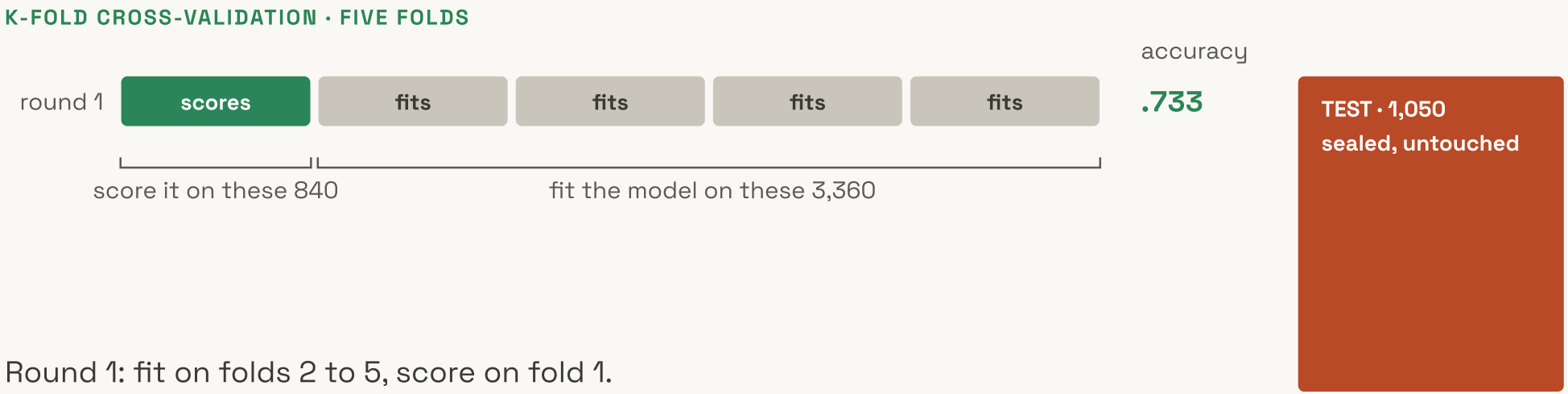


Cut the training data into five equal **folds**.

Each fold will sit out once, while the other four fit the model.

< 3 / 8 >

Other designs



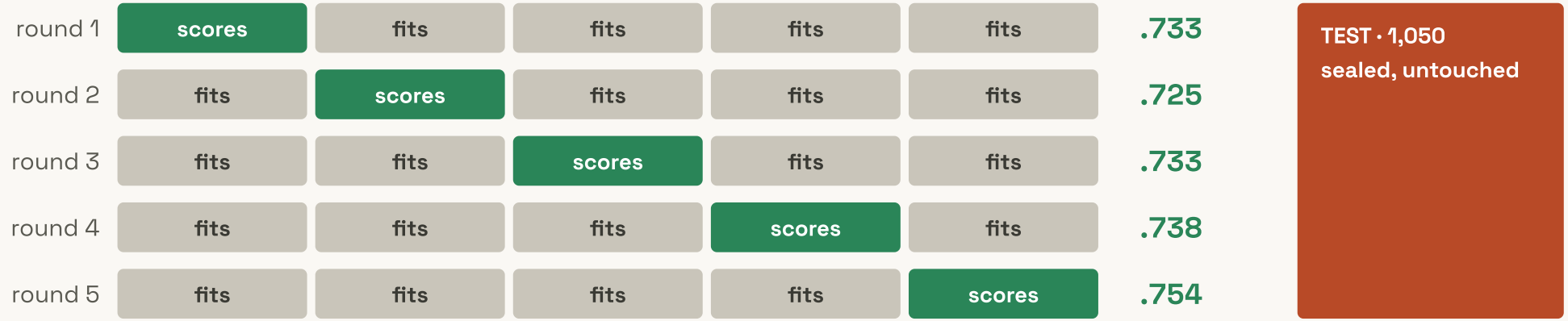
Round 1: fit on folds 2 to 5, score on fold 1.

The model has not seen fold 1, so its score there is honest.

< 4 / 8 >

Other designs

K-FOLD CROSS-VALIDATION · FIVE FOLDS

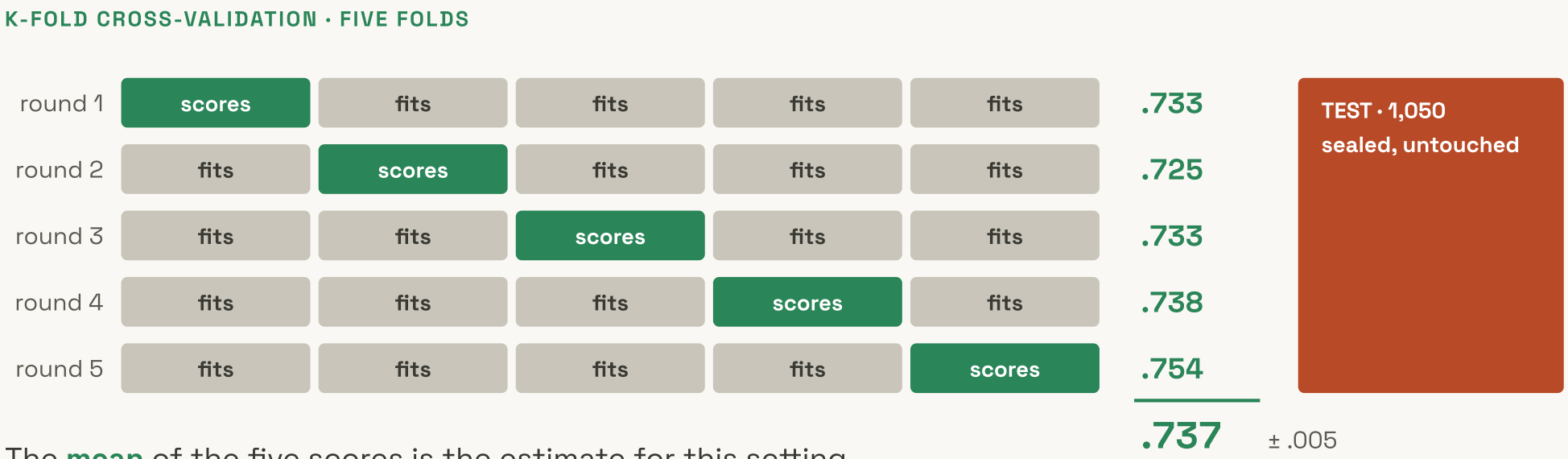


Rounds 2 to 5: every fold sits out once.

Every abstract is scored once, by a model that did not see it.

< 5 / 8 >

Other designs



The **mean** of the five scores is the estimate for this setting.

Their spread says how far to trust it: here ± .005.

Other designs

ONE MEAN PER SETTING · KEEP THE BEST

the network, 16 hidden units

penalty	mean of five folds
0.0001	.685 ± .037
0.01	.728 ± .002
0.3	**.751 ± .007**
1	.647 ± .009

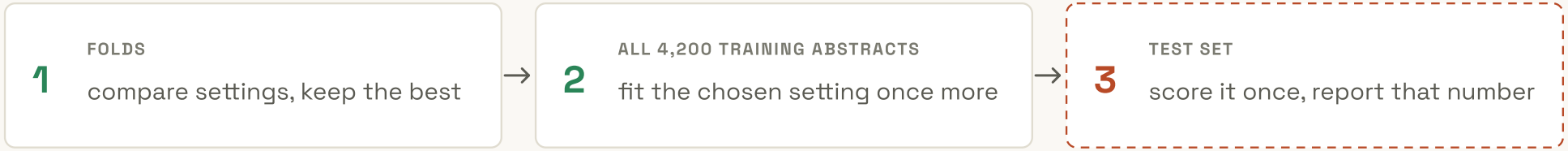
Run the five rounds once per setting, and keep the best mean.

The tuning in Part V does this for 294 settings.

< 7 / 8 >

Other designs

WHERE THE NUMBERS GO



The test set sees one model, once. Nothing else about it is ever computed.

< 8 / 8 >

Testing is validation

- The test set carries the human labels. Scoring the model on it is exactly last week's comparison of an instrument with a gold standard
- Same confusion matrix, same accuracy, precision, recall, F1 and κ, per class, with the base rate beside them
- The only new thing: the labels the model is scored on were **not** used to build it. Last week that was automatic, because nobody trains a dictionary
- Report the test-set score, the size of the test set, and the class frequencies; without them nobody can judge the score

> **Everything in Part III of last week applies unchanged; only the instrument is new.**

III · A classifier, end to end

Three abstracts: which discipline?

ARXIV PREPRINT

Reversible temperature exchange upon thermal contact

Abstract. According to a well-known principle of thermodynamics, the transfer of heat between two bodies is reversible when their temperatures are infinitesimally close. As we demonstrate, a little-known alternative exists: two bodies with temperatures different by an arbitrary amount can completely exchange their temperatures in a reversible way if split into infinitesimal parts that are brought into thermal contact sequentially.

physics

ARXIV PREPRINT

Singing Style Transfer Using Cycle-Consistent Boundary Equilibrium Generative Adversarial Networks

Abstract. Can we make a famous rap singer like Eminem sing whatever our favorite song? Singing style transfer attempts to make this possible, by replacing the vocal of a song from the source singer to the target singer. This paper presents a method that learns from unpaired data for singing style transfer using generative adversarial networks.

computer science

ARXIV PREPRINT

Adversarial Attacks and Defences Competition

Abstract. To accelerate research on adversarial examples and robustness of machine learning classifiers, Google Brain organized a NIPS 2017 competition that encouraged researchers to develop new methods to generate adversarial examples as well as to develop new ways to defend against them. In this chapter, we describe the structure and organization of the competition and the solutions developed by several of the top-placing teams.

statistics

The data: 5,250 abstracts, three disciplines

```
science <- read_csv("data/science-abstracts.csv") |>
  mutate(label = factor(label))

science |> select(label, title, text)

science |> count(label)
```

- Titles and abstracts of arXiv papers, labelled by the section they were submitted to
- The task: predict the discipline from the words, a proxy for any topic classification you would do on news or parliamentary speech
- 1,750 per class, so class imbalance is not in the way today; it will be in yours

OUTPUT

```
# A tibble: 5,250 × 3
  label            title                  text
  <fct>            <chr>                  <chr>
1 computer science Deep Learning Micro... Deep Learning M...
2 statistics       The Reduced PC-Algo... The Reduced PC-...
3 computer science Critique of Barbosa... Critique of Bar...
4 physics          Gate-error analysis... Gate-error anal...
5 statistics       On synthetic data w... On synthetic da...
# ℹ 5,245 more rows
```

```
# A tibble: 3 × 2
  label               n
  <fct>           <int>
1 computer science  1750
2 physics           1750
3 statistics        1750
```

Abstracts from arXiv.org, labelled by primary category; compiled for the 2024 edition of this course. Licence: arXiv non-exclusive distribution.

The workflow

split	> recipe	> model	> workflow	> fit	> predict	> evaluate
`initial_split()`	`recipe()`	`multinom_reg()`	`workflow()`	`fit()`	`predict()`	`conf_mat()`
hold part back	from text to features	which algorithm, which settings	recipe and model, as one object	on the training data	on the test data	against the test labels

- The next slides take these seven steps one at a time; the row of chips above each slide shows where we are
- Together they are steps 02 to 04 of the pipeline, written as code

> **The recipe, the model and the workflow are objects you can print, share and reuse. Swapping the algorithm changes one of them.**

Kuhn, M., & Silge, J. (2022). *Tidy modeling with R*. O'Reilly. https://www.tmwr.org

VRIJE UNIVERSITEIT AMSTERDAM

Split first

```
1  library(tidymodels)
2
3  set.seed(42)                                  # the split is reproducible
4  split <- initial_split(science, prop = .8,    # 80 / 20
5                         strata = label)        # each class keeps its share
6
7  train_data <- training(split)
8  test_data  <- testing(split)
9
10 tibble(set = c("training", "test"),
11        n = c(nrow(train_data), nrow(test_data)))
```

OUTPUT

```
# A tibble: 2 × 2
  set          n
  <chr>    <int>
1 training  4200
2 test      1050
```

```
# A tibble: 3 × 2
  label               n
  <fct>           <int>
1 computer science   350
2 physics            350
3 statistics         350
```

- Nothing below this slide touches `test_data` until the confusion matrix
- One thousand and fifty held-out abstracts: the fresh two hundred, made real

VU VRIJE UNIVERSITEIT AMSTERDAM

The recipe

```
rec <- recipe(label ~ text, data = train_data) |>   # predict label from text
  step_tokenize(text) |>                            # 1 · words
  step_stopwords(text) |>                           # 2 · drop the grammar
  step_tokenfilter(text, min_times = 20,            # 3 · the 1,000 most frequent
                   max_tokens = 1000) |>            #     words seen 20+ times
  step_tf(text)                                     # 4 · the document–term matrix

baked <- rec |> prep() |> bake(new_data = NULL)
baked |> select(label, tf_text_data, tf_text_quantum) |> print(n = 5)
```

OUTPUT

```
# A tibble: 4,200 × 3
  label            tf_text_data tf_text_quantum
  <fct>                   <int>           <int>
1 computer science            0               0
2 computer science            0               0
3 computer science            0               0
4 computer science            1               0
5 computer science            0               0
# ℹ 4,195 more rows
```

- A **recipe** writes the preprocessing down before it runs: learned on the training data, applied unchanged to the test data
- Steps 1 and 2 are the tokens and stop words of weeks 1 and 2; step 3 is week 2's trimming, and 1,000 columns is a **hyperparameter**
- `prep()` learns the vocabulary, `bake()` applies it: 4,200 rows, one column per kept word

split recipe model workflow fit predict evaluate

Specify and fit

```
logit_spec <- multinom_reg(penalty = 0.01,    # shrink the weights ...
                           mixture = 1) |>    # ... lasso: many go to zero
  set_engine("glmnet") |>                     # the R package that fits it
  set_mode("classification")

logit_wf <- workflow() |>
  add_recipe(rec) |>                          # the recipe from before
  add_model(logit_spec)                       # the model from above

m_logit <- fit(logit_wf, data = train_data)
```

- `rec` is the recipe from the last four slides, unchanged. The workflow binds it to the model
- Logistic regression for three classes: a weight per word per class, summed and turned into three probabilities
- The **penalty** is the answer to Part II: it punishes large weights, so a thousand features cannot memorise 4,200 abstracts

> **Fitting took 11 seconds on this laptop.**

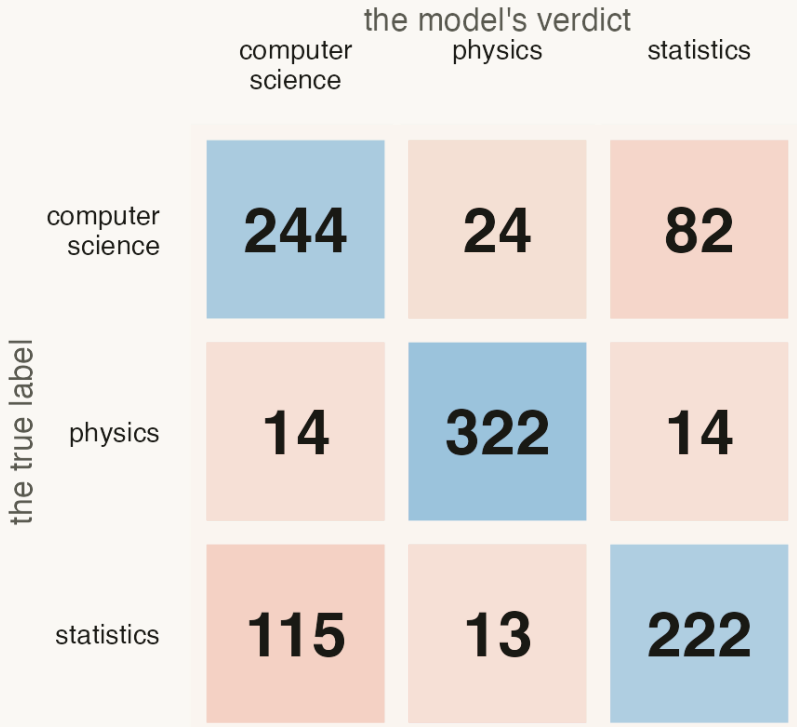
VU VRIJE UNIVERSITEIT AMSTERDAM

The confusion matrix, three by three

```
pred_logit <- predict(m_logit, new_data = test_data) |>
  bind_cols(test_data |> select(id, label, title))

pred_logit |>
  conf_mat(truth = label, estimate = .pred_class)
```

- The model has never seen these 1,050 abstracts. Rows are the truth, columns the verdict; the diagonal is what it got right
- The errors sit between statistics and computer science: *model*, *data*, *learning* belong to both



> **Every off-diagonal cell is a pair of disciplines that share a vocabulary. The matrix shows how often the model fails and where the construct is fuzzy.**

split recipe model workflow fit predict evaluate

The numbers you report

```
class_metrics <- metric_set(accuracy, precision,
                            recall, f_meas)

pred_logit |>
  class_metrics(truth = label, estimate = .pred_class)

pred_logit |>                          # per true class
  group_by(label) |>
  accuracy(truth = label, estimate = .pred_class)
```

OUTPUT

```
# A tibble: 4 × 3
  .metric   .estimator .estimate
  <chr>     <chr>          <dbl>
1 accuracy  multiclass     0.750
2 precision macro          0.750
3 recall    macro          0.750
4 f_meas    macro          0.749
```

```
# A tibble: 3 × 4
  label            .metric  .estimator .estimate
  <fct>            <chr>    <chr>          <dbl>
1 computer science accuracy multiclass     0.697
2 physics          accuracy multiclass     0.92
3 statistics       accuracy multiclass     0.634
```

- Three classes, so precision and recall are averaged: **macro** by default, every class counting equally
- Per class, the model finds 92% of the physics abstracts but only 70% of computer science and 63% of statistics. The average hides which construct it cannot separate

> **Three abstracts in four, on texts the model never saw. Whether that is good depends on what an error costs in the study.**

split recipe model workflow fit predict evaluate

VU VRIJE UNIVERSITEIT AMSTERDAM

What the model gets wrong

ARXIV PREPRINT

Comparative Study of Virtual Machines and Containers for DevOps Developers

Abstract. In this work, we plan to develop a system to compare virtual machines with container technology. We would devise ways to measure the administrator effort of containers vs. Virtual Machines (VMs). Metrics that will be tested against include human efforts required, ease of migration, resource utilization and ease of use using containers and virtual machines.

filed under: computer science

the model: physics

ARXIV PREPRINT

TherML: Thermodynamics of Machine Learning

Abstract. In this work we offer a framework for reasoning about a wide class of existing objectives in machine learning. We develop a formal correspondence between this work and thermodynamics and discuss its implications.

filed under: statistics

the model: computer science

ARXIV PREPRINT

On the generation of the quarks through spontaneous symmetry breaking

Abstract. In this paper we present the state of the art about the quarks: group SU(3), Lie algebra, the electric charge and mass. The quarks masses are generated in the same way as the lepton masses. It is constructed a term in the Lagrangian that couples the Higgs doublet to the fermion fields.

filed under: physics

the model: computer science

> Every number on the last two slides is made of cases like these. Read some before you report the score.

VRIJE UNIVERSITEIT AMSTERDAM

Swap the algorithm: one object changes

```
library(discrim)

nb_spec <- naive_Bayes(Laplace = 1) |>        # add one to every count
  set_engine("naivebayes", usepoisson = TRUE) |>  # the inputs are counts
  set_mode("classification")

nb_wf <- workflow() |>
  add_recipe(rec) |>                          # the same recipe
  add_model(nb_spec)                          # a different model

m_nb <- fit(nb_wf, data = train_data)

pred_nb <- predict(m_nb, new_data = test_data) |>
  bind_cols(test_data |> select(id, label))
```

- **Naive Bayes** is a dictionary the data wrote: a weight per word and class, from the counts
- The recipe, the split and the metrics are untouched: only the model specification is new

THE DICTIONARY IT WROTE: EACH CLASS'S MOST DISTINCTIVE WORDS

COMPUTER SCIENCE	PHYSICS	STATISTICS
robot	planets	nonparametric
speech	orbital	gene
communication	redshift	multivariate
social	stellar	regression
mobile	fermi	likelihood
language	spin	estimators
web	stars	bayesian

PLAYGROUND Let the data write the dictionary

split recipe model workflow fit predict evaluate

VU VRIJE UNIVERSITEIT AMSTERDAM

Swap again: a support vector machine

```
rec_norm <- rec |>
  step_normalize(all_predictors())        # SVMs want scaled inputs

svm_wf <- workflow() |>
  add_recipe(rec_norm) |>                 # the recipe, plus one step
  add_model(svm_linear() |>               # a different model
              set_engine("LiblineaR") |>
              set_mode("classification"))

m_svm <- fit(svm_wf, data = train_data)

pred_svm <- predict(m_svm, new_data = test_data) |>
  bind_cols(test_data |> select(id, label))
```

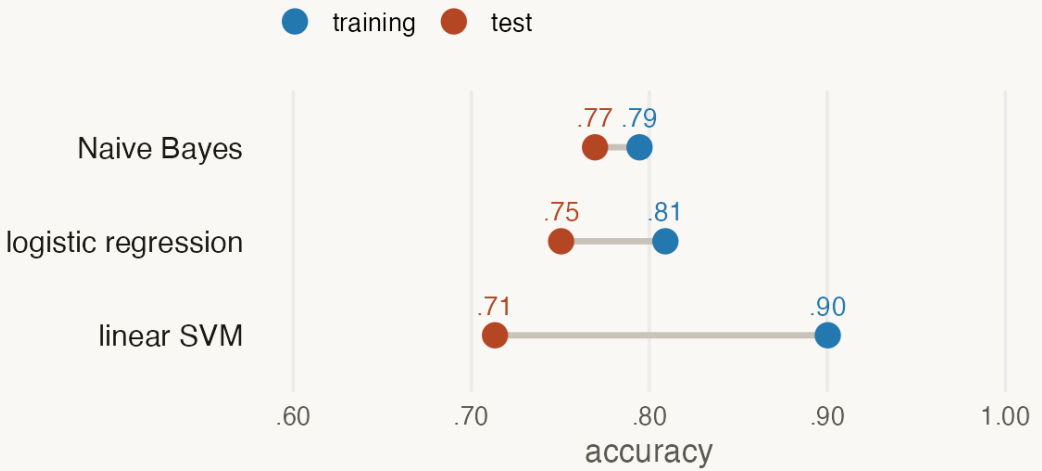
- The recipe gains one step, because an SVM wants scaled inputs; the split and the metrics are untouched
- A linear support vector machine draws the widest-margin boundary through the thousand-dimensional space of word counts
- This is what the workflow buys: the comparison is fair, because everything else is held fixed

Three algorithms, one test set

```
accuracy_on <- function(model, data) {
  predict(model, new_data = data) |>
    bind_cols(select(data, label)) |>
    accuracy(truth = label, estimate = .pred_class) |>
    pull(.estimate)
}

models <- list(`logistic regression` = m_logit,
               `Naive Bayes` = m_nb, `linear SVM` = m_svm)

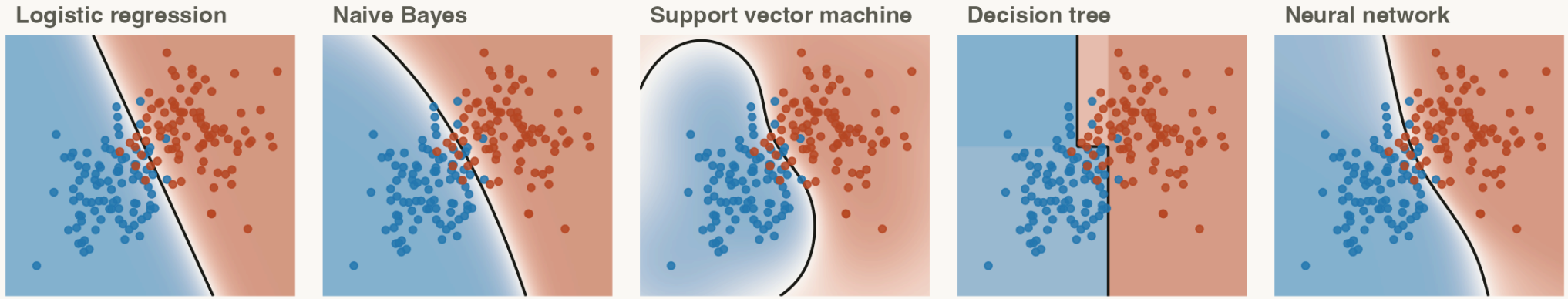
tibble(model    = names(models),
       training = map_dbl(models, accuracy_on, train_data),
       test     = map_dbl(models, accuracy_on, test_data)) |>
  pivot_longer(-model, names_to = "set", values_to = "accuracy") |>
  ggplot(aes(accuracy, model, colour = set)) +
  geom_point(size = 4)
```



- The SVM fits its training abstracts best and the test abstracts worst: Part II's ninth-degree polynomial, on real data

> Training accuracy favours the most flexible model. The ranking that counts comes from the test set, the same one for all three.

Which algorithm?



The three you just ran, plus a tree and a network, on the same 180 points. The line is where the verdict flips; the tint is how sure the model is.

> Each model draws its boundary in a different shape. Text has a thousand features instead of two: the same picture, impossible to draw.

Which algorithm, in words

ALGORITHM	THE RULE IT LEARNS	STRONG WHEN	WEAK WHEN
Logistic regression	one weight per word, summed	many features; readable weights	meaning lies in combinations
Naive Bayes	word rates per class, multiplied	little data; a fast baseline	words are correlated
Support vector machine	the widest-margin boundary	sparse, high-dimensional text	you need probabilities
Decision tree	yes/no questions on words	mixed feature types	text: brittle, overfits
Neural network	weighted sums, stacked, non-linear	combinations matter; lots of data	little data; nothing readable

> **For a bag of words, start with Naive Bayes, then try a regularised logistic regression or a linear SVM.**

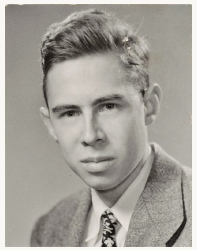
Hvitfeldt, E., & Silge, J. (2021). *Supervised machine learning for text analysis in R*. CRC Press. https://smltar.com — Chapters 6–7.

IV · Neural networks

Sixty-eight years of one idea

1958 — The perceptron

- Frank Rosenblatt's first trainable **neural network**: weighted inputs, a threshold, a learning rule
- Minsky & Papert (1969) show what one layer cannot learn; neural nets go out of fashion



1966 — The first dictionary: the General Inquirer

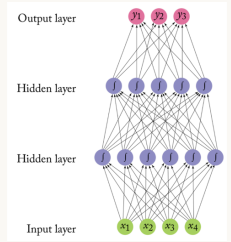
- Stone, Dunphy, Smith & Ogilvie (Harvard): word lists per category, applied to texts by a mainframe — the first computer **content analysis**, and the **dictionary approach** of week 2
- Same year: Weizenbaum's ELIZA imitates a therapist by pattern matching; the ALPAC report ends US funding for machine translation

1970s–80s — Knowledge-based NLP

- SHRDLU (1970) follows commands in a blocks world; expert systems
- Meaning is encoded by hand in ontologies: concepts, properties, relations
- Works in toy worlds, does not scale to real text

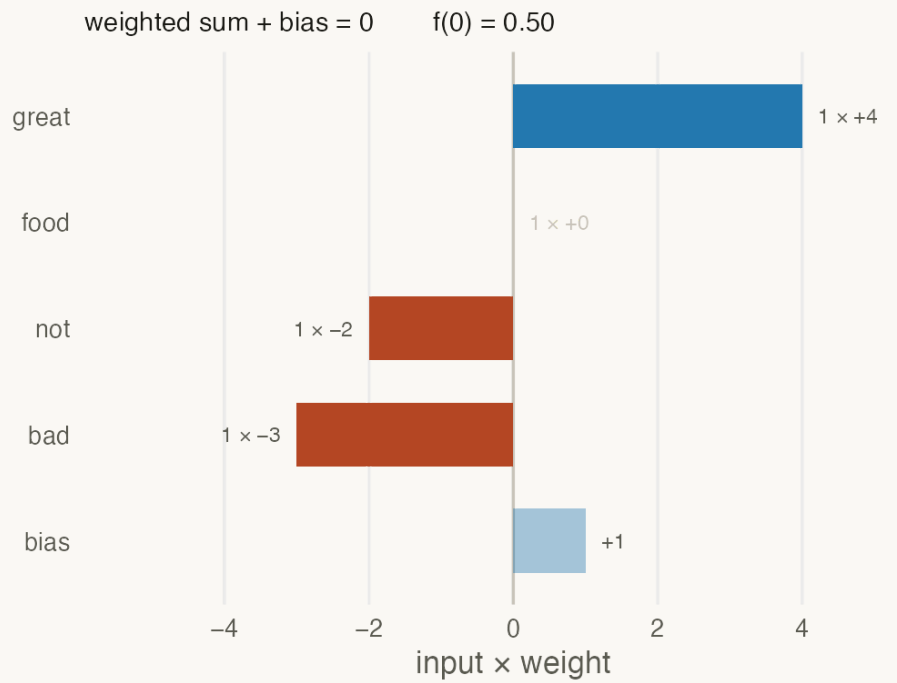
1986 — Backpropagation

- Rumelhart, Hinton & Williams: multi-layer networks can be trained
- Neural nets return — but data and compute are still too scarce for language



From the neuron to the unit

- A biological neuron sums the signals it receives and fires once a threshold is passed. That is where the analogy starts and ends
- The artificial **unit** (McCulloch & Pitts, 1943; Rosenblatt, 1958): each input x is multiplied by a **weight** w, the products are summed, a **bias** is added, and an **activation function** turns the sum into an output
- Written out: output = $f(w_1x_1 + w_2x_2 + \ldots + b)$: the same weighted sum as last week's word list, drawn on the right
- A **network** stacks units in layers: the outputs of one layer are the inputs of the next; a unit only talks to the layer before and the layer after



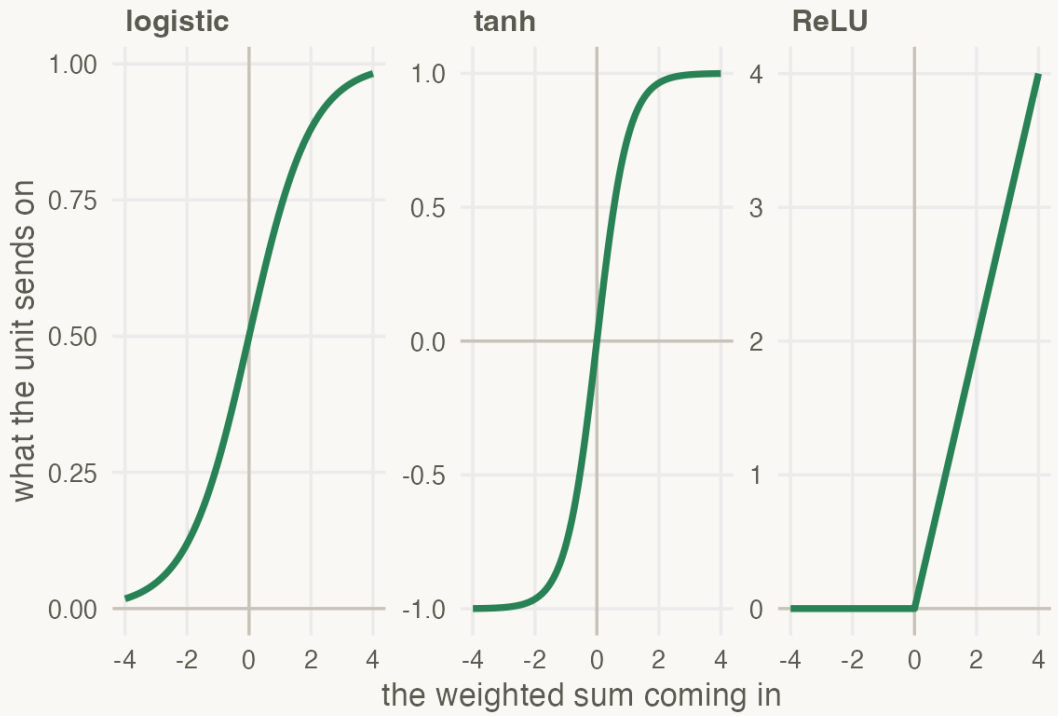
ONE REVIEW, ONE UNIT

> **One unit is a regression. A network is regressions whose inputs are other regressions, and that is the only difference.**

Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review, 65*(6), 386–408. https://doi.org/10.1037/h0042519

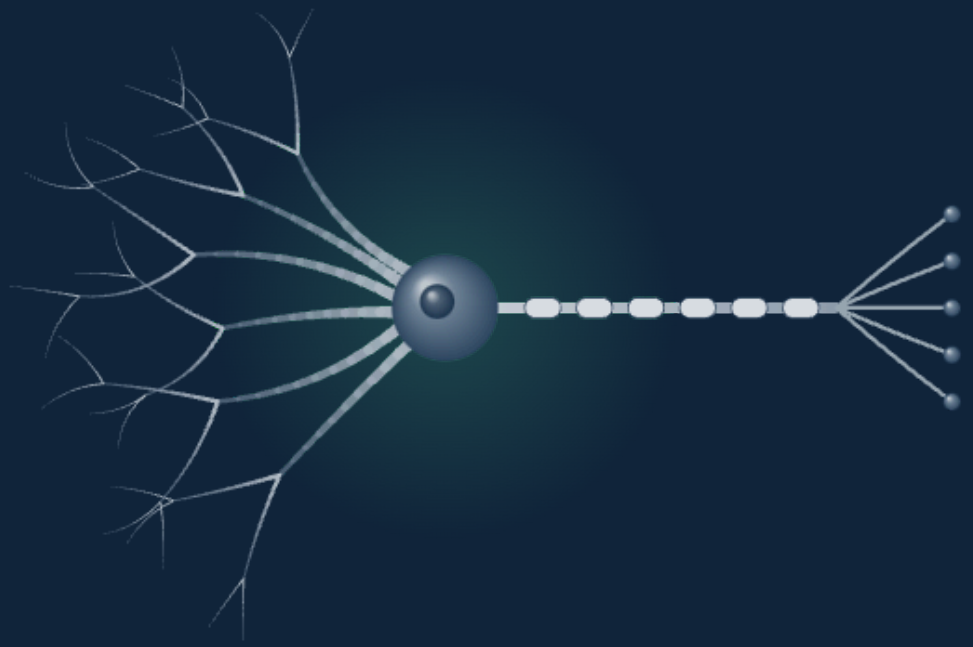
The activation function

- Without it, a stack of weighted sums is one weighted sum: the hidden layer would add nothing
- The **logistic** curve squashes any sum into (0, 1), a probability: with it, one unit is a logistic regression
- **tanh** has the same S-shape between −1 and +1; the network on the next slide uses it inside
- **ReLU**, zero below zero and the sum itself above, is the workhorse inside large modern networks



The network, taken apart

A NEURON

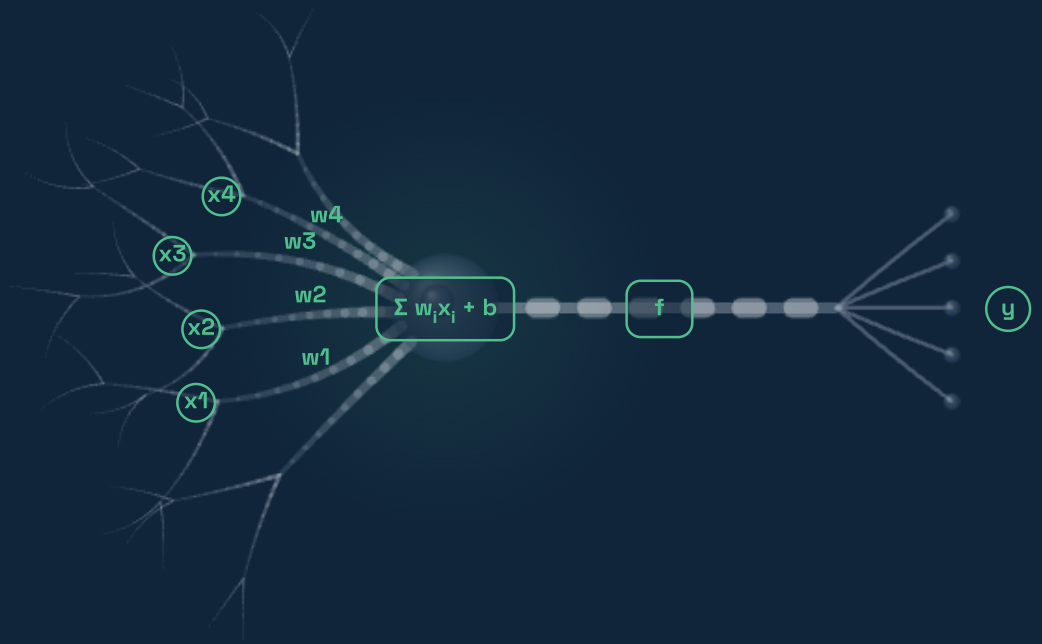


FROM THE BRAIN

- A neuron collects signals, adds them up, and fires once they pass a threshold
- That is as far as the comparison goes

The network, taken apart

ONE UNIT



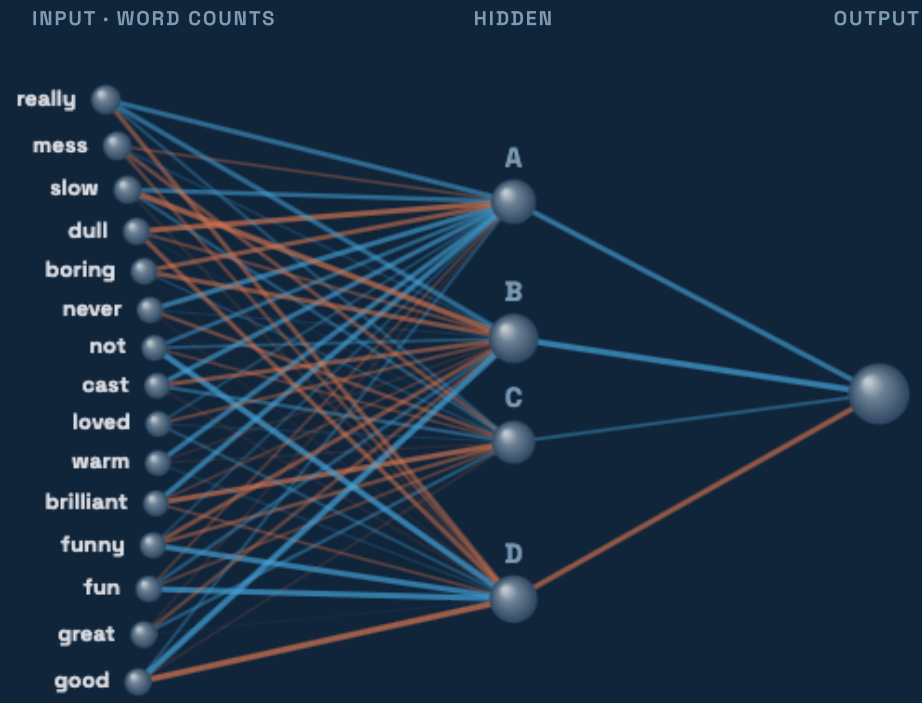
ONE UNIT

- Each input **x** is multiplied by a weight **w**; the results are added up, plus a bias **b**
- An S-curve **f**, the **activation function**, turns the sum into the output
- With the logistic curve as **f**, one unit is a **logistic regression**

$$y = f(w_1x_1 + w_2x_2 + \ldots + b)$$

< 2 / 17 >

The network, taken apart

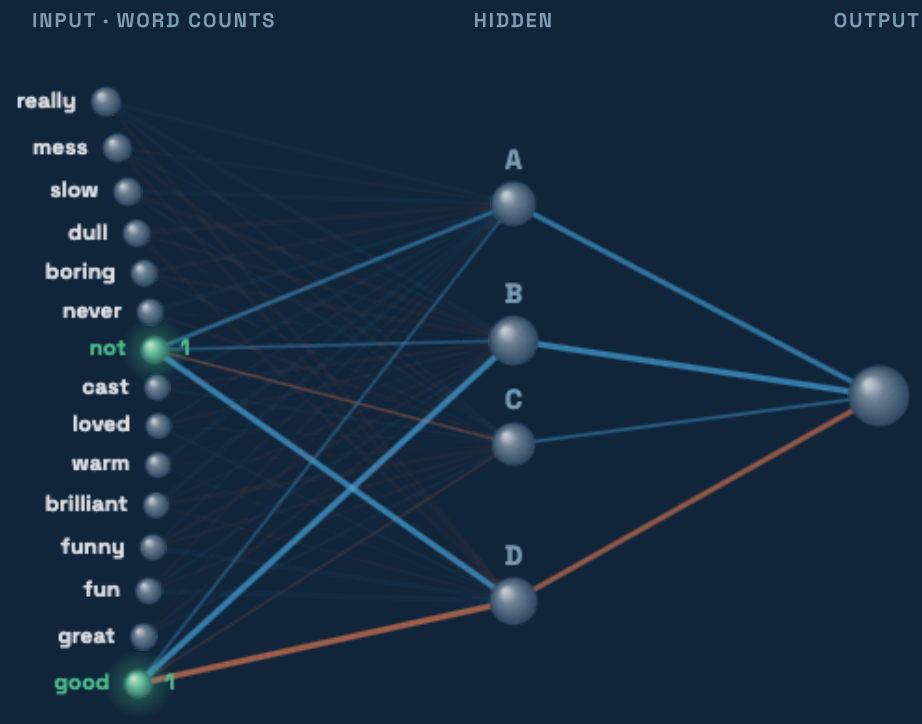


A NETWORK OF UNITS

- Units in layers: what one layer sends out, the next takes in
- Here: 15 words in, 4 hidden units **A** to **D**, 1 output
- The cold open's network, before training: the weights are random

< 3 / 17 >

The network, taken apart

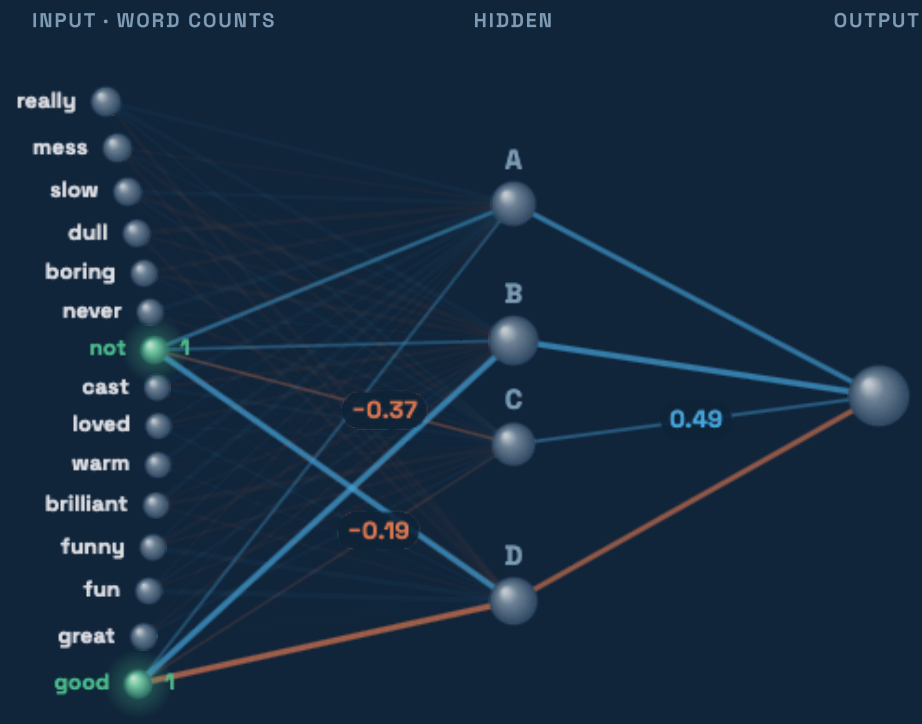


The review: **“Not good at all.”** human label: **negative**

ONE REVIEW GOES IN

- **“Not good at all.”** The human label is **negative**
- The network sees word counts: **not** 1, **good** 1, the other 13 words 0

The network, taken apart

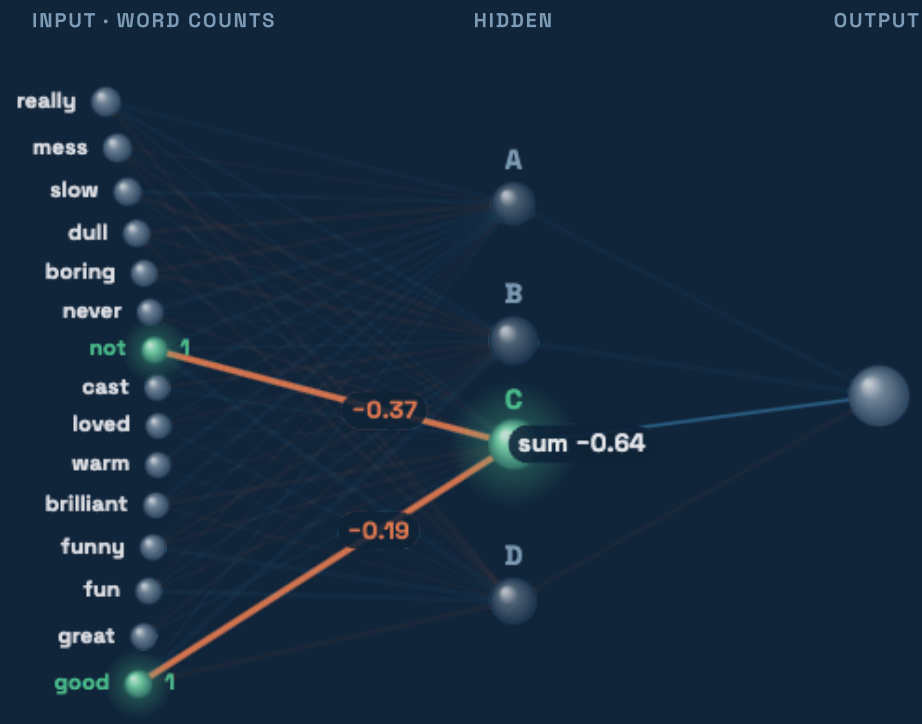


The review: **“Not good at all.”** human label: **negative**

THE WEIGHTS

- Every line is a weight: one number
- **Blue** positive, **orange** negative, thicker when larger
- We follow unit **C.** Its lines carry the numbers shown

The network, taken apart



The review: **“Not good at all.”** human label: **negative**

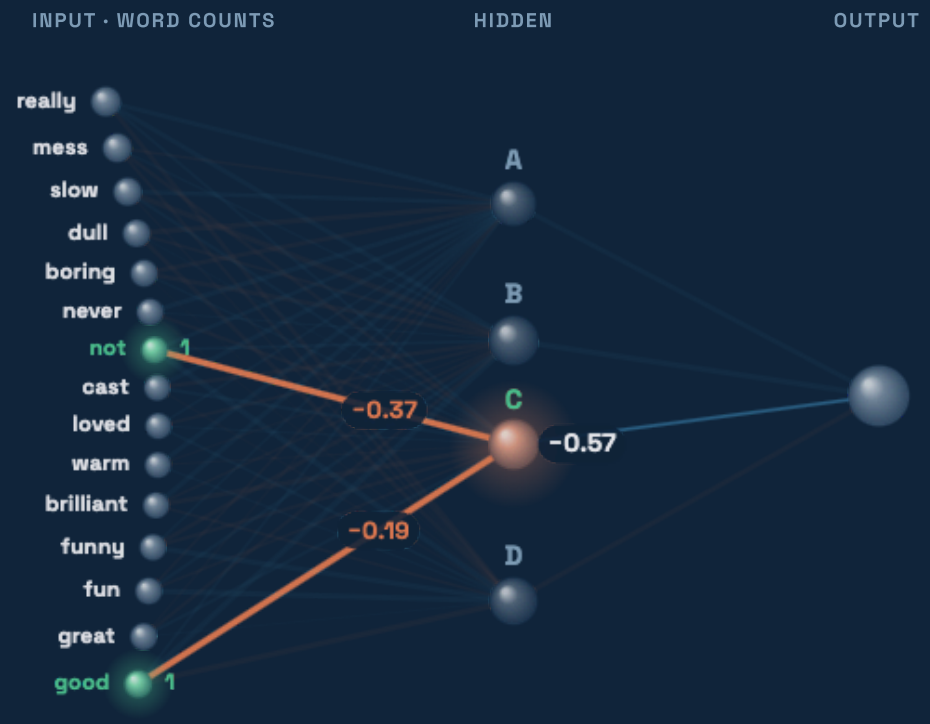
UNIT C ADDS UP

- Each count times its weight, then C’s own number, the **bias**

not	1 × −0.37	**= −0.37**
good	1 × −0.19	**= −0.19**
bias		**−0.08**
sum		**−0.64**

- Words with a count of 0 add nothing

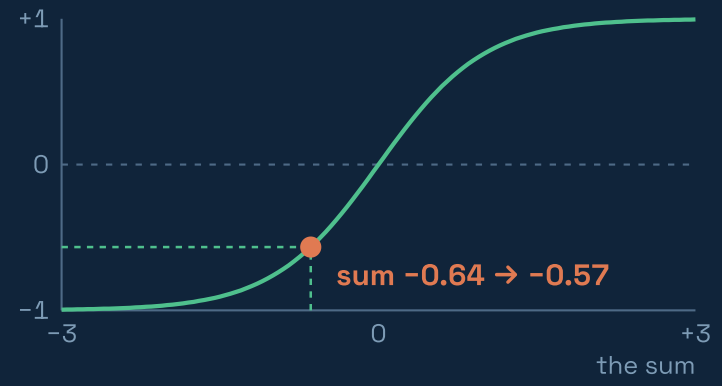
The network, taken apart



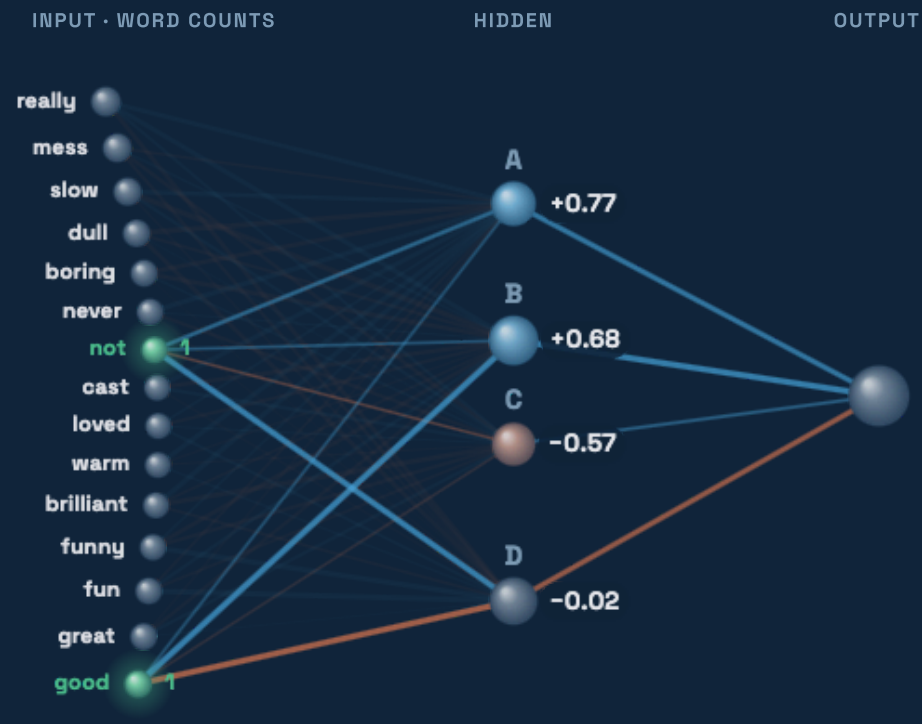
The review: **“Not good at all.”** human label: **negative**

UNIT C SQUASHES THE SUM

- The activation function, here **tanh**, is an S-curve
- Whatever the sum, the output stays between −1 and +1



The network, taken apart



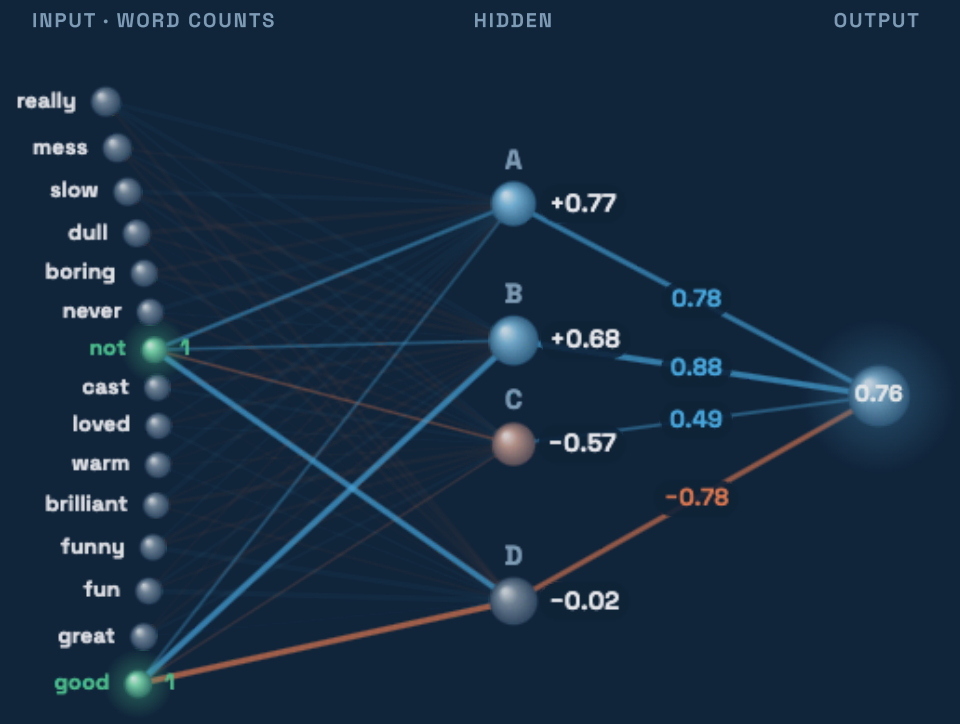
The review: **“Not good at all.”** human label: **negative**

ALL FOUR UNITS

- **A**, **B** and **D** do the same, each with its own weights
- Four numbers leave the hidden layer

A	+0.77
B	+0.68
C	−0.57
D	−0.02

The network, taken apart



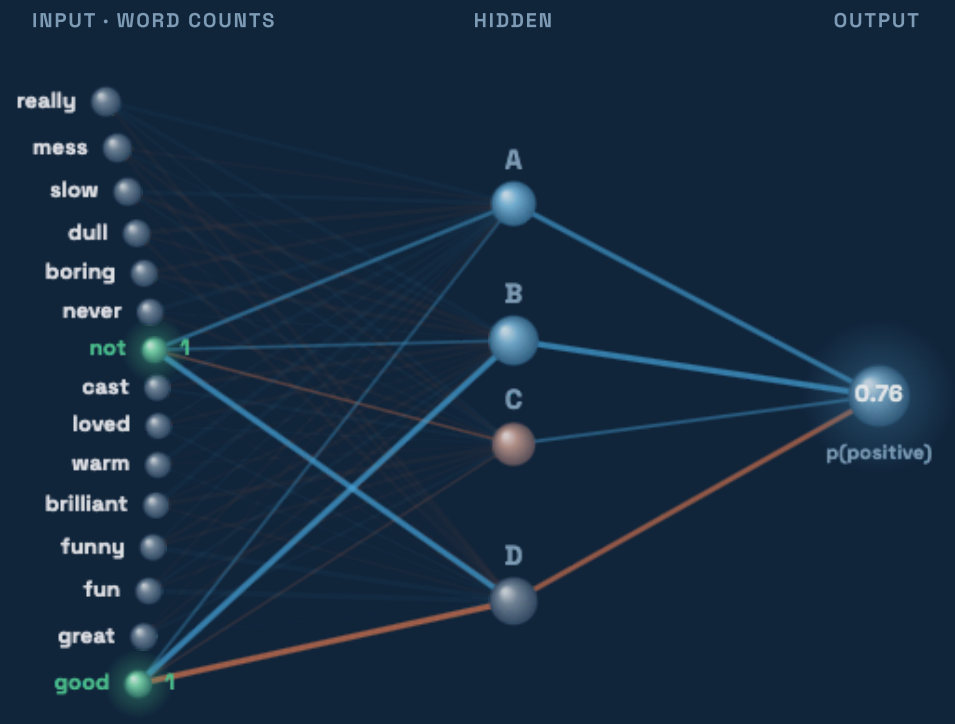
The review: **“Not good at all.”** human label: **negative**

THE OUTPUT UNIT ADDS UP

- Same rule: each unit’s number times its weight, plus a bias

A	+0.77 × +0.78	**= +0.60**
B	+0.68 × +0.88	**= +0.60**
C	−0.57 × +0.49	**= −0.28**
D	−0.02 × −0.78	**= +0.02**
bias		**+0.23**
sum		**+1.17**

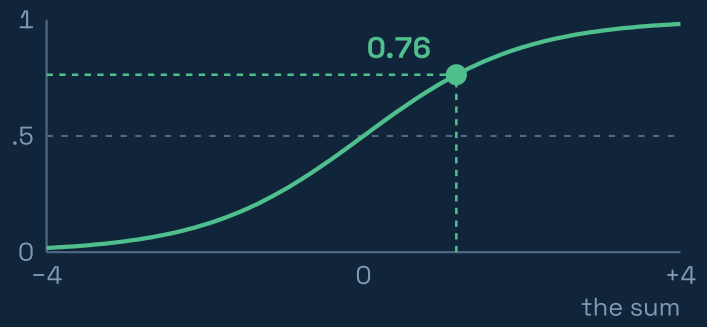
The network, taken apart



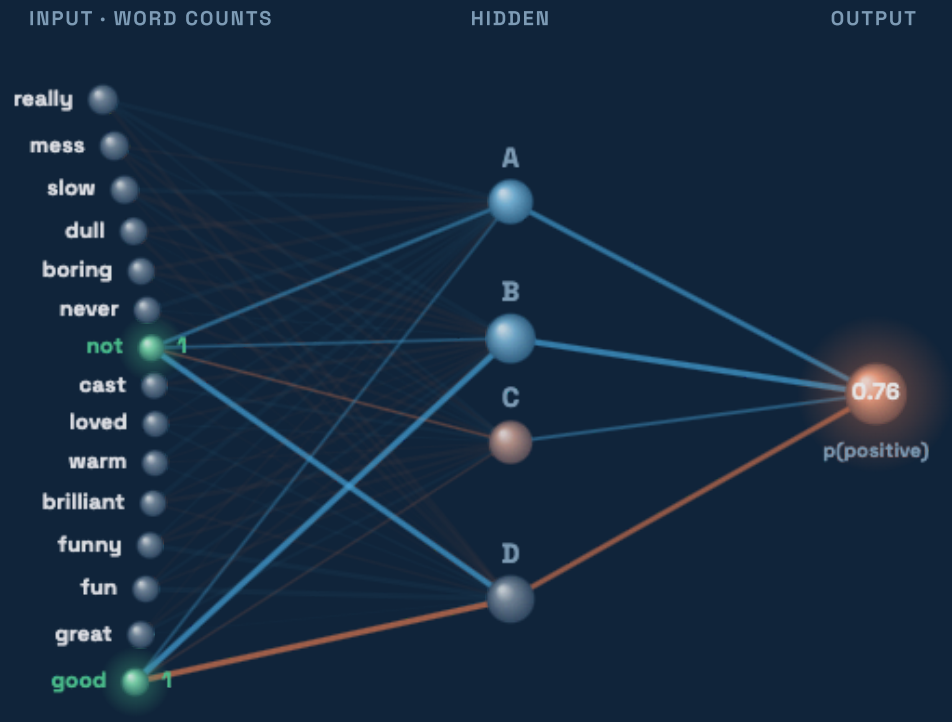
The review: **“Not good at all.”** human label: **negative**

… AND MAKES IT A PROBABILITY

- The logistic curve turns the sum into a number between 0 and 1
- **p(positive) = 0.76**: the network says positive, the human said negative
- The output unit is a **logistic regression** on A to D



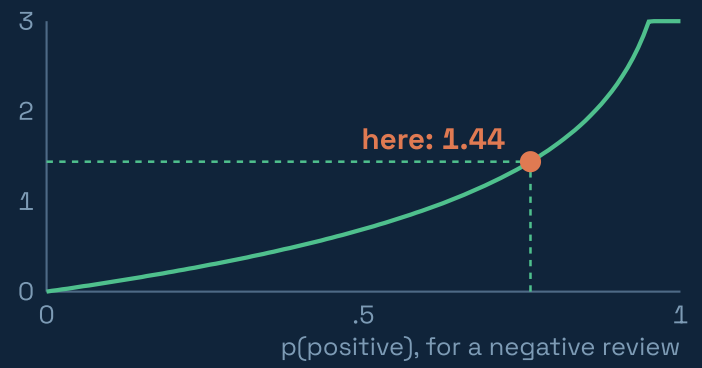
The network, taken apart



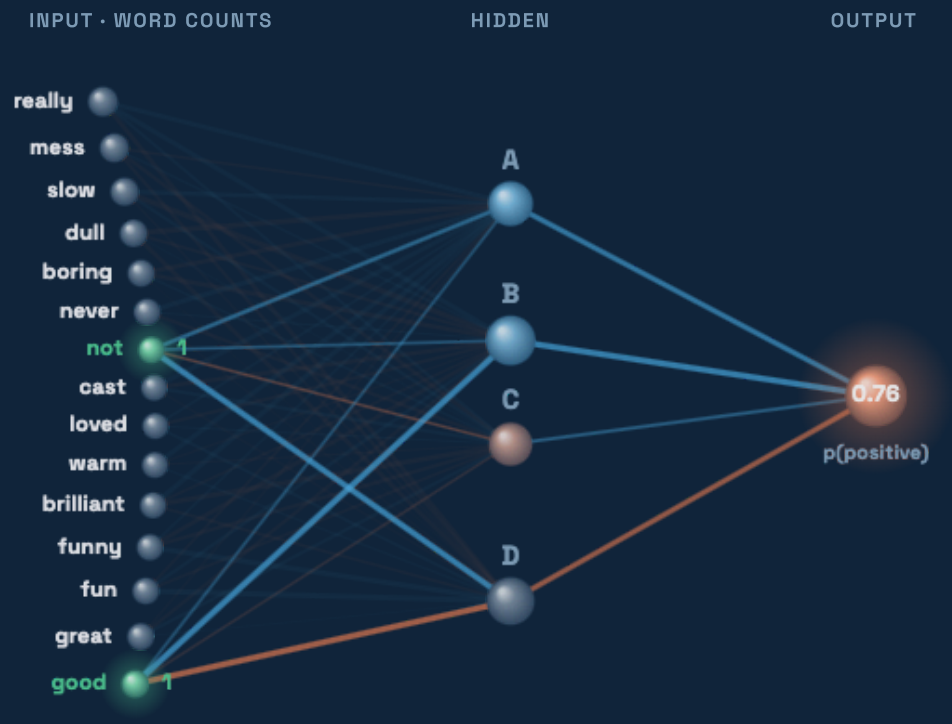
The review: **“Not good at all.”** human label: **negative**

HOW WRONG? THE LOSS

- The **loss** turns a miss into one number
- Near 0 when the answer is right and sure, large when it is wrong and sure



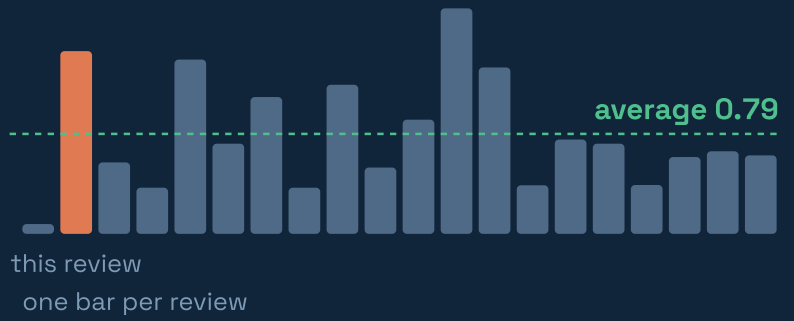
The network, taken apart



The review: **“Not good at all.”** human label: **negative**

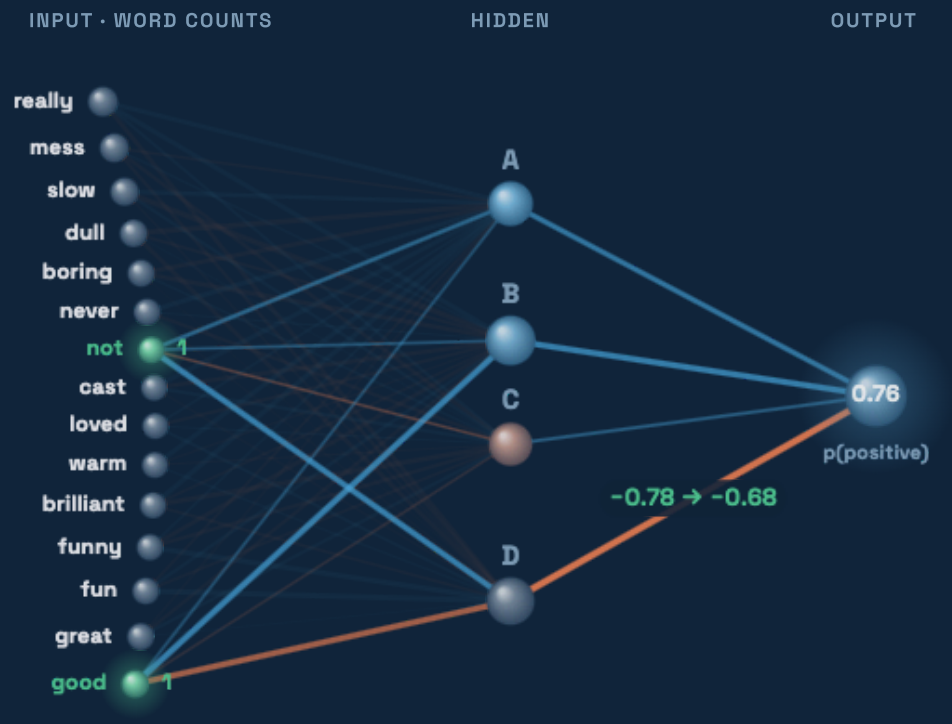
ALL TWENTY REVIEWS

- Every review gets its own loss
- Training changes the weights so that their **average** gets smaller. Now: 0.79



< 12 / 17 >

The network, taken apart

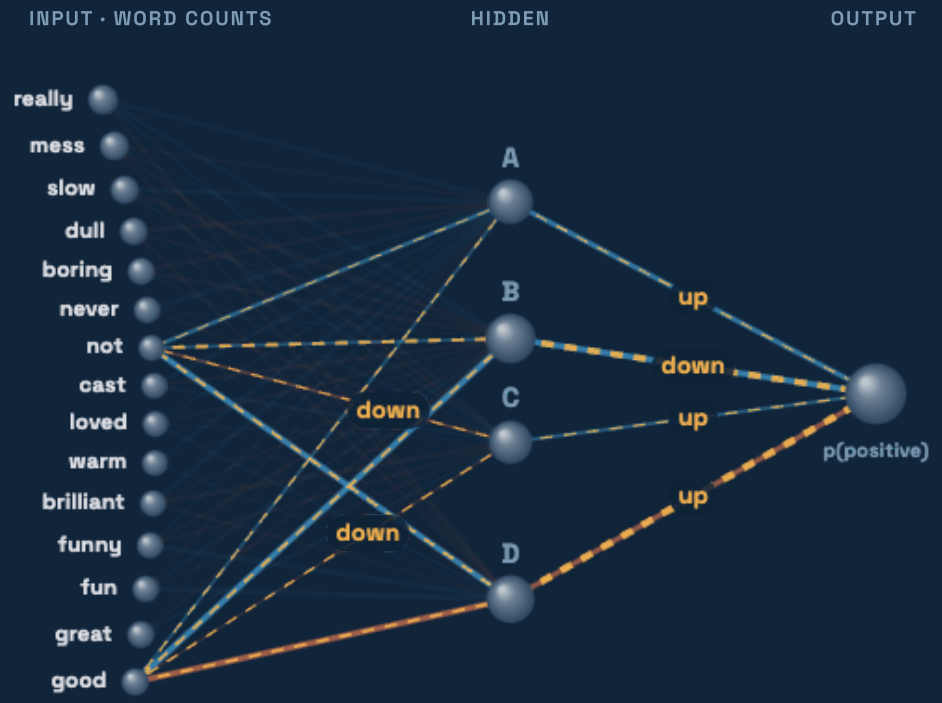


The review: **“Not good at all.”** human label: **negative**

WHICH WAY SHOULD A WEIGHT MOVE?

- Take one weight, **D’s line to the output**, and make it 0.1 larger
- The average loss over the twenty: 0.788 → 0.777
- It falls, so this weight should go up. Had it risen, it should go down

The network, taken apart

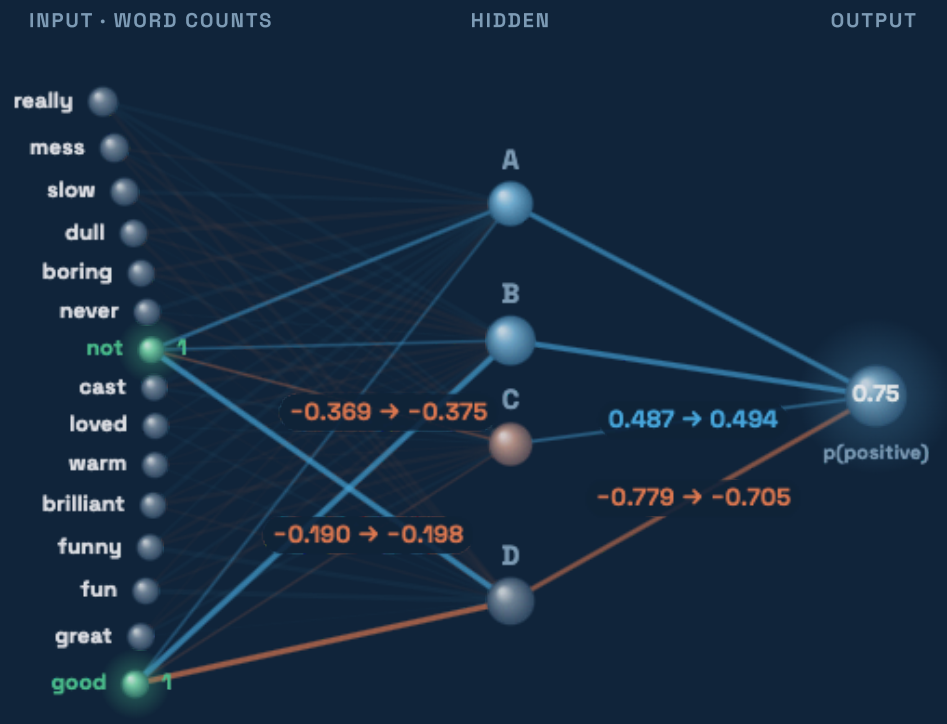


The review: **“Not good at all.”** human label: **negative**

BACKPROPAGATION

- Asking this for all 64 weights, one at a time, would be slow
- **Backpropagation** answers for all of them at once, working back from the output
- Amber: which way each weight should go, and the thicker, the more it matters now

The network, taken apart

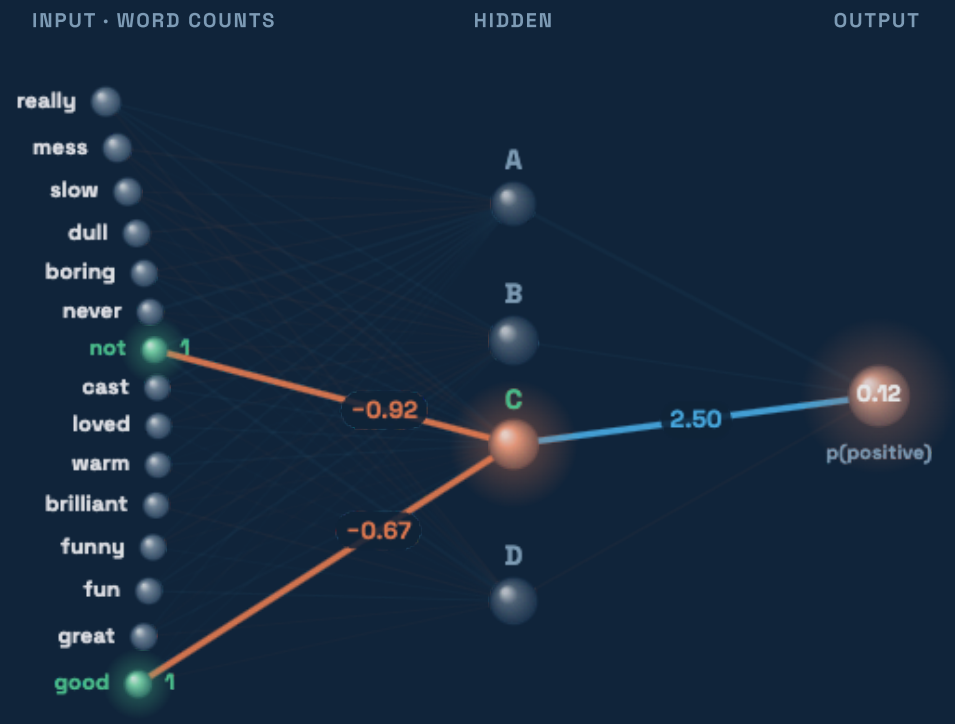


ONE STEP

- Every weight takes a small step in its direction
- How big a step: the **learning rate**, set by you
- This review: p(positive) 0.763 → 0.751
- Average loss: 0.788 → 0.756

The review: **“Not good at all.”** human label: **negative**

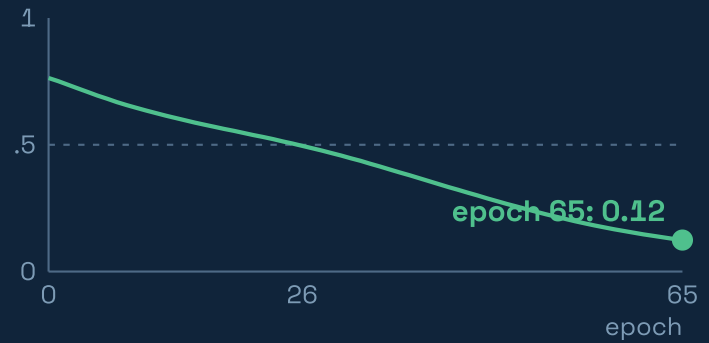
The network, taken apart



The review: **“Not good at all.”** human label: **negative**

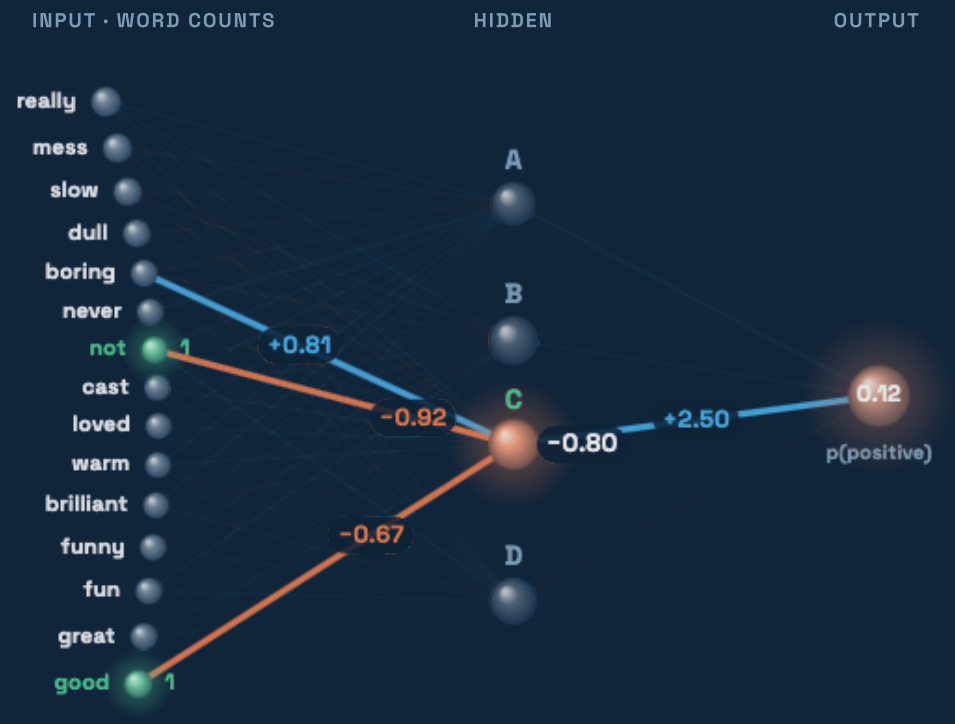
AGAIN AND AGAIN

- One pass over all twenty, then one step: an **epoch**
- Watch C’s lines grow, and this review’s p(positive) fall



Epoch 26: this review comes out right
Epoch 65: all twenty do

The network, taken apart



The review: **“Not good at all.”** human label: **negative**

WHAT UNIT C LEARNED

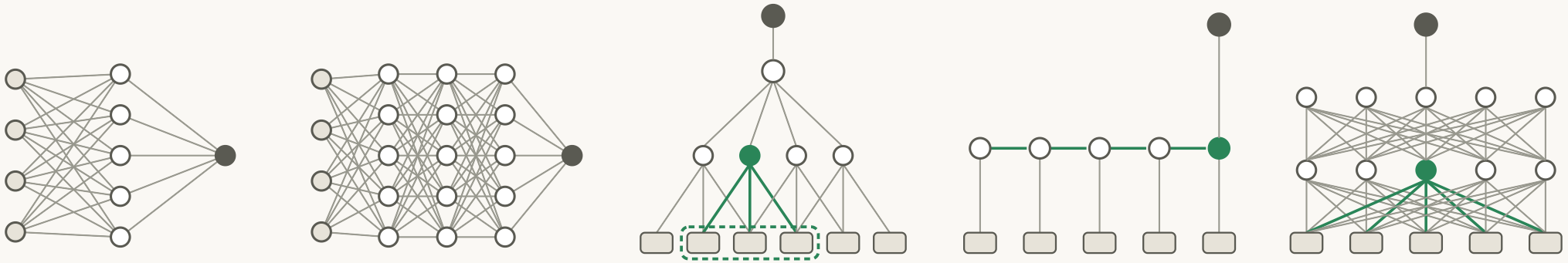
- **“Not good at all.”**: C sends **−0.80**, toward negative
- **“Not boring at all.”**: C sends **+0.38**, toward positive
- The two differ only in **good** and **boring**. Nobody told C to tell them apart; its weights moved there because the loss fell

What a hidden unit can see that a list cannot

- A dictionary gives *good* +1 whatever stands in front of it; Naive Bayes gives *good* one learned weight, and the same problem
- A hidden unit can learn a weight on *not* **and** a weight on *good* and fire only when both are present: a combination, not a sum
- *Not good at all* is the first text in this course that a model gets right without being told the rule
- The cheap alternative from week 1 still stands: put *not good* in the matrix as a bigram, and the list works again. The network learns the bigram; the tokeniser declares it

> **This is the specific thing the hidden layer buys: features made of features. Next week's models do this with many layers, at scale.**

Other ways to wire the units



One hidden layer
this week

Many hidden layers
deep learning

Convolutional
a window slides over words

Recurrent
reads in order, remembers

Transformer · week 4
words look at each other

> All of them are units with weights, trained the same way, by backpropagation. What differs is how the units are wired.

VU VRIJE UNIVERSITEIT AMSTERDAM

Fitting a network with tidymodels

```
nn_spec <- mlp(hidden_units = 16,       # one hidden layer, 16 units
               epochs = 100,            # passes over the training set
               learn_rate = 0.1,        # the step size
               penalty = 0.01) |>       # the brake on the weights
  set_engine("brulee") |>               # torch, inside R
  set_mode("classification")

nn_wf <- workflow() |>
  add_recipe(rec_norm) |>               # scaled inputs, as for the SVM
  add_model(nn_spec)

set.seed(42)                            # the random starting weights
m_nn <- fit(nn_wf, data = train_data)

pred_nn <- predict(m_nn, new_data = test_data) |>
  bind_cols(test_data |> select(id, label))
```

- A multilayer perceptron: 1,000 inputs, 16 hidden units, 3 outputs (about sixteen thousand weights)
- Every number in the specification is a hyperparameter; none has a right value in advance
- The seed matters now in two places: the split, and the random initial weights

> **Fitting took 2 seconds. The recipe, the split and the metrics did not change.**

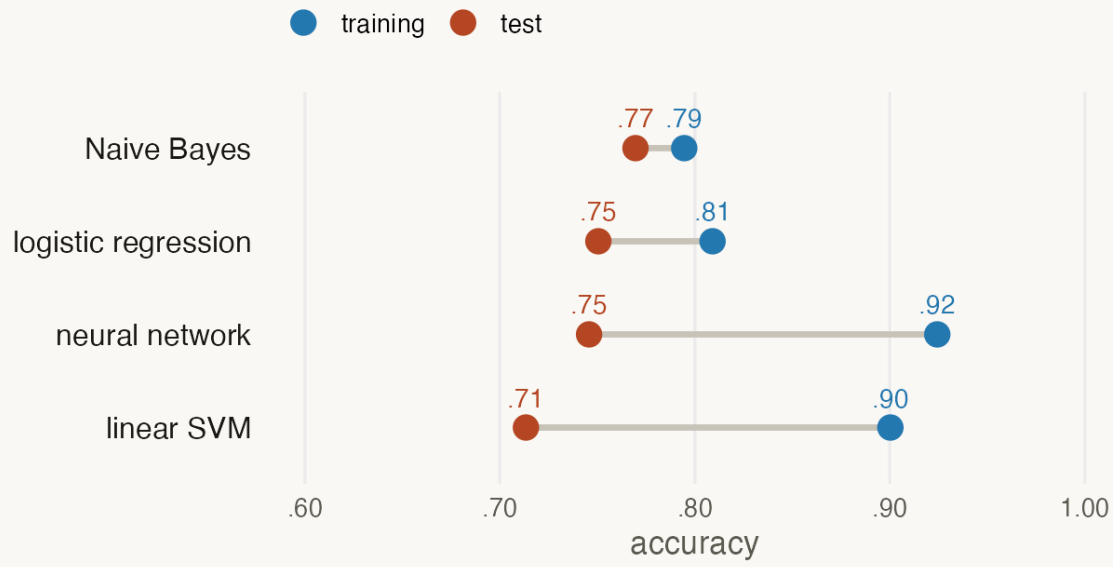
Kuhn, M., & Falbel, D. (2023). *brulee: High-level modeling functions with torch* (R package). https://CRAN.R-project.org/package=brulee

split recipe model workflow fit predict evaluate

Four algorithms, one test set

- On the test set the network lands among the three models from Part III
- On its own training abstracts it reaches .92, the widest gap of the four: sixteen thousand weights for 4,200 abstracts
- Where a network pays off is when the *input* carries structure worth combining, and that is what the next part changes

> **The representation limits the score more than the algorithm.**



V · Word embeddings

From text to numbers, again

01 characters | 02 tokens | 03 counts | 04 embeddings

TEXT 1 To be or not to be

TEXT 2 What does it mean to be

WEEKS 1–3

Counts

one row per text, one column per word

2	2	1	1	0	0
1	1	0	0	1	1
0	3	1	0	2	0

WEEKS 3–4

Embeddings

one column per word, 300 rows nobody named

.21	−.48	.77	.05
−.34	.19	.62	−.41
.07	.55	−.15	.66

Playground · the tokeniser lets you build the left one yourself

Two shapes carry the whole course.

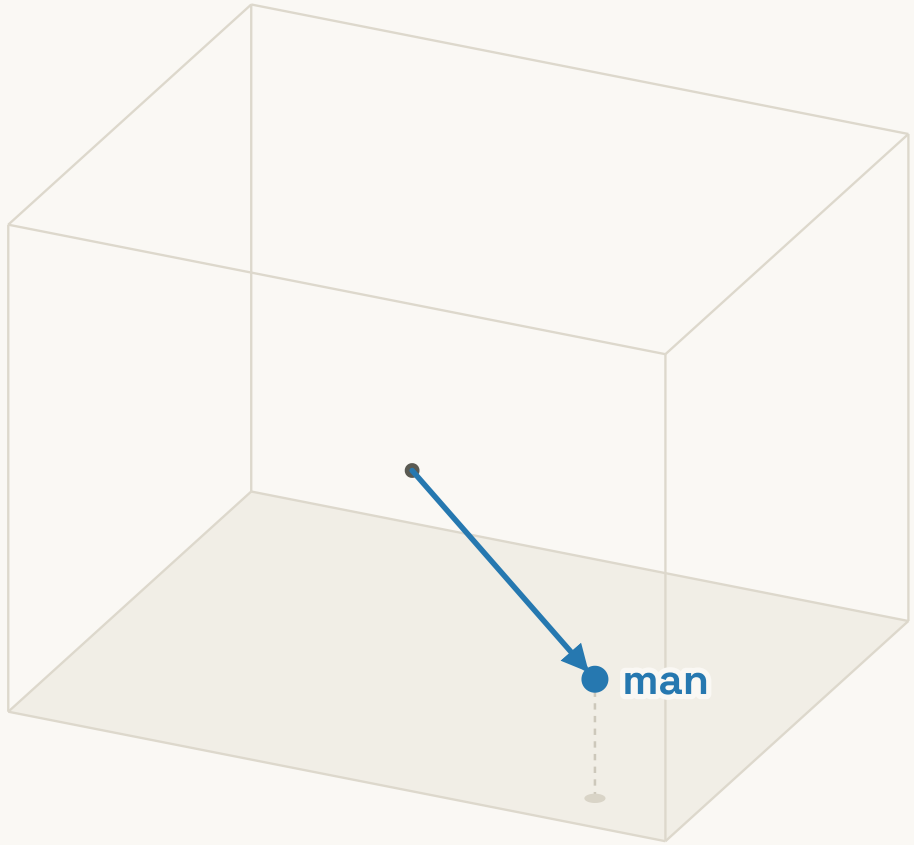
5 / 5

What a bag of words cannot see

- Every model so far took the document–term matrix as its input: one column per word, and every column a stranger to every other
- *fantastic* and *great* share nothing in that matrix. A model that learned *great* from the training set knows nothing about a test abstract that says *fantastic*
- The matrix is **sparse** (94 cells in 100 were zero last week) and **wide**: a thousand columns for a thousand words, and the 1,001st word gets none
- Word order is gone, and with it *not good*; the hidden unit could only rebuild it because both words survived the cut

> **The representation throws away similarity between words before the model ever sees them. So the fix goes into the representation.**

Words as vectors



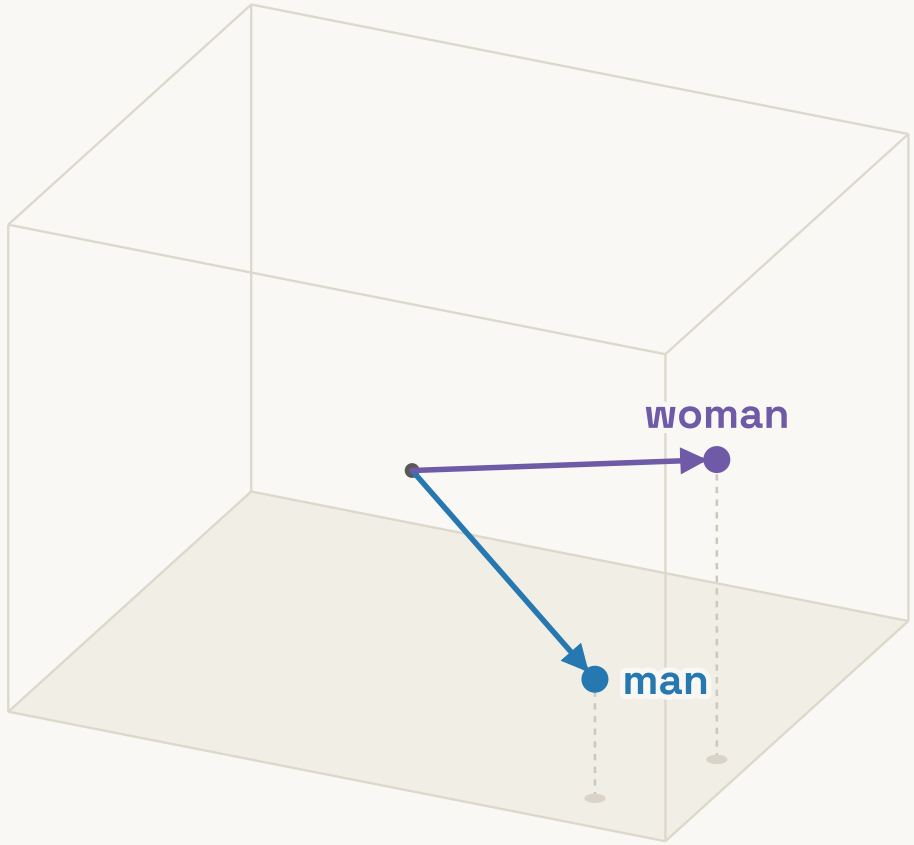
EACH WORD: 50 NUMBERS

man



1 / 8

Words as vectors



EACH WORD: 50 NUMBERS

man

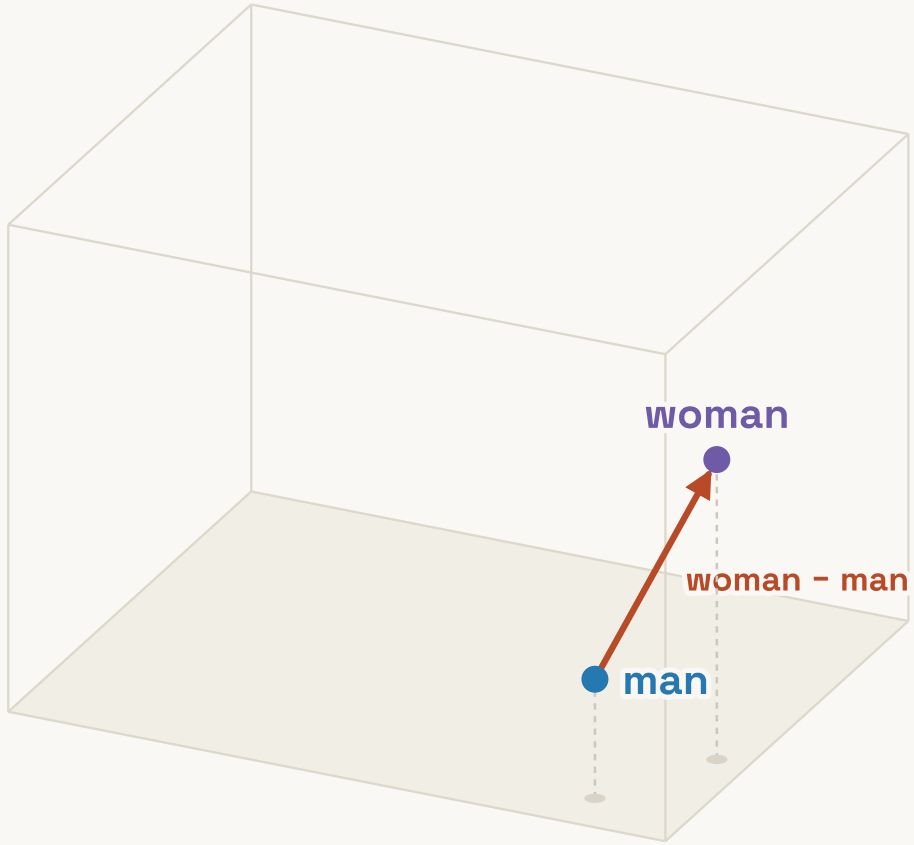


woman



2 / 8

Words as vectors



EACH WORD: 50 NUMBERS

man



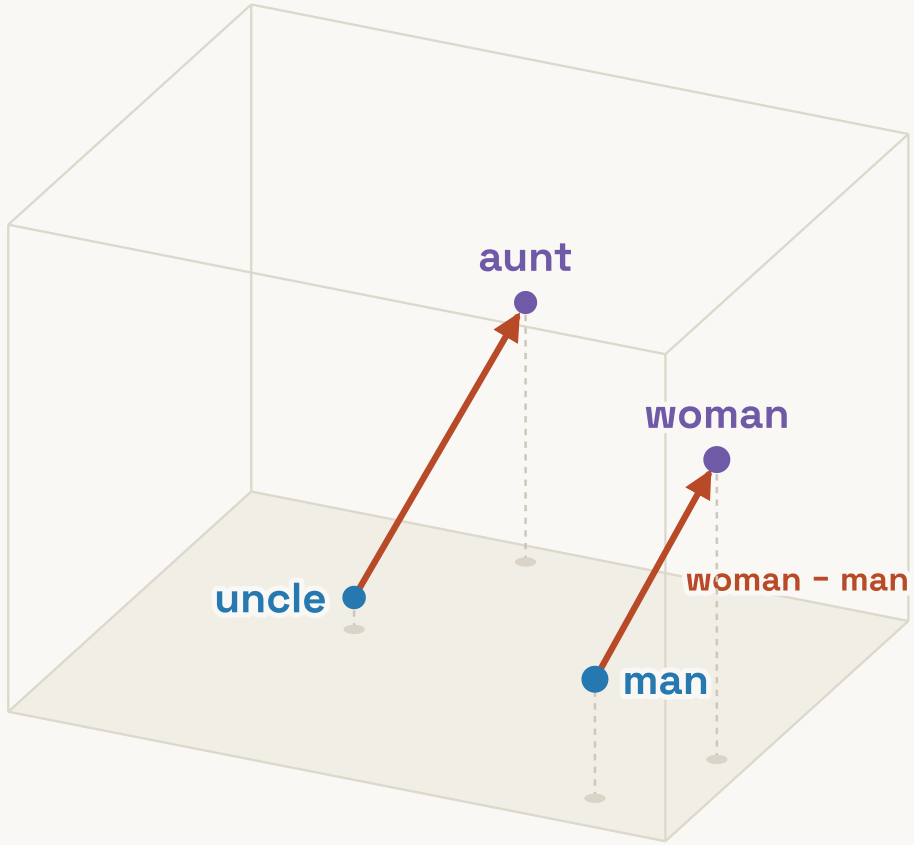
woman



woman − man



Words as vectors



EACH WORD: 50 NUMBERS

man



woman



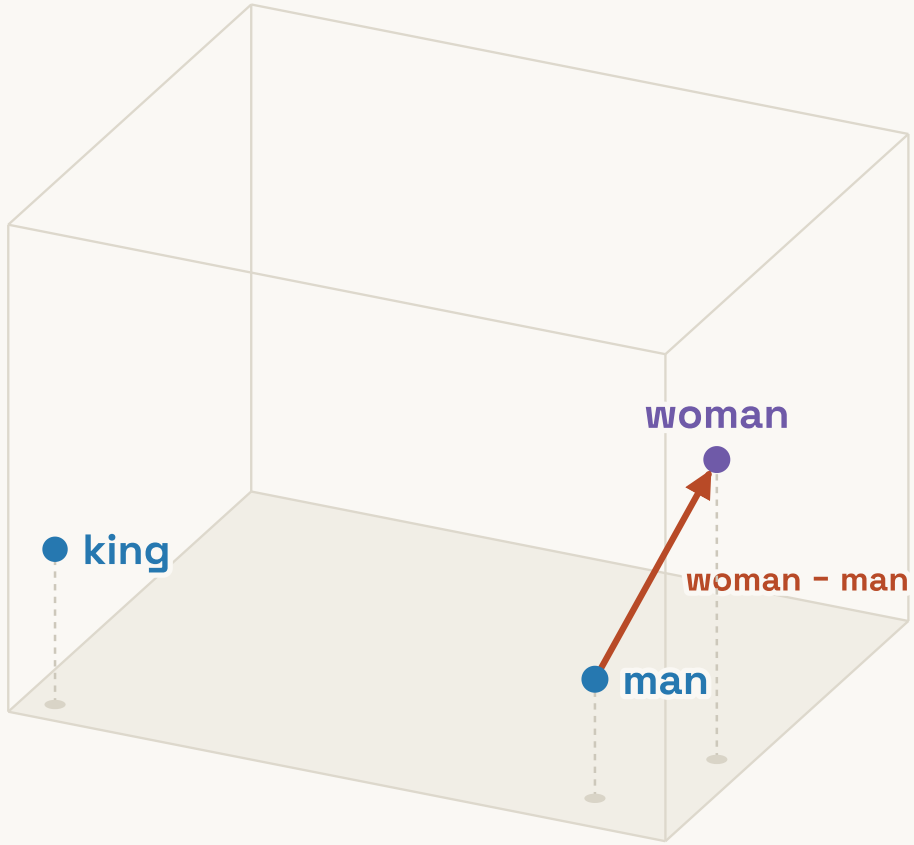
woman - man



aunt - uncle



Words as vectors



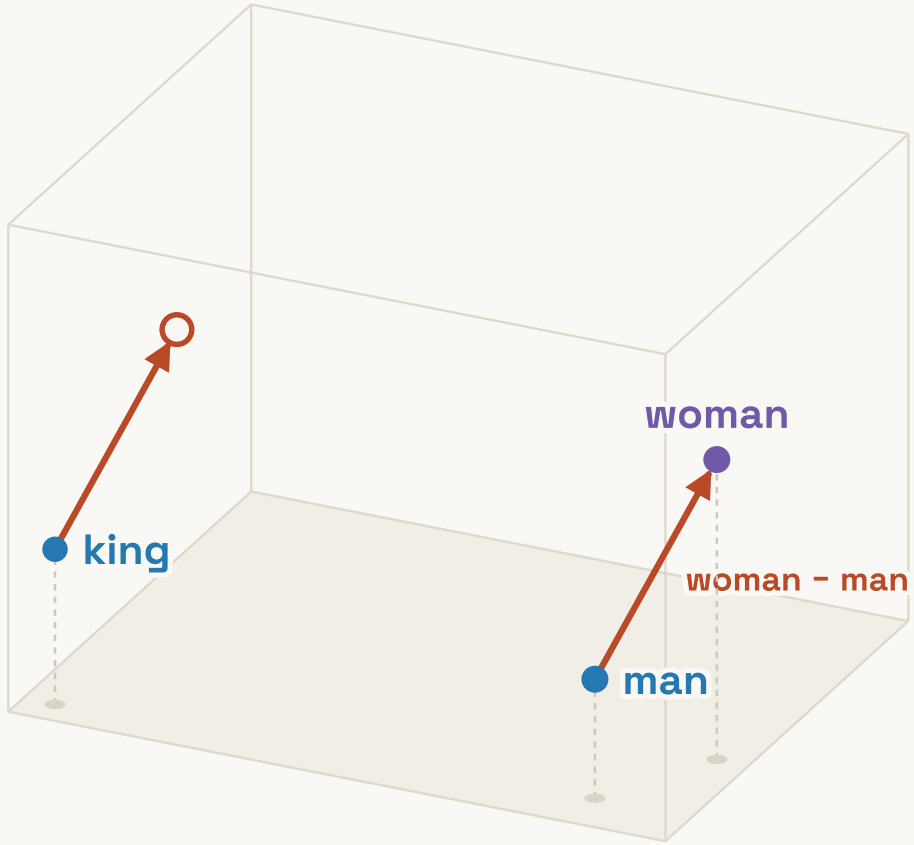
king − man + woman

king



5 / 8

Words as vectors



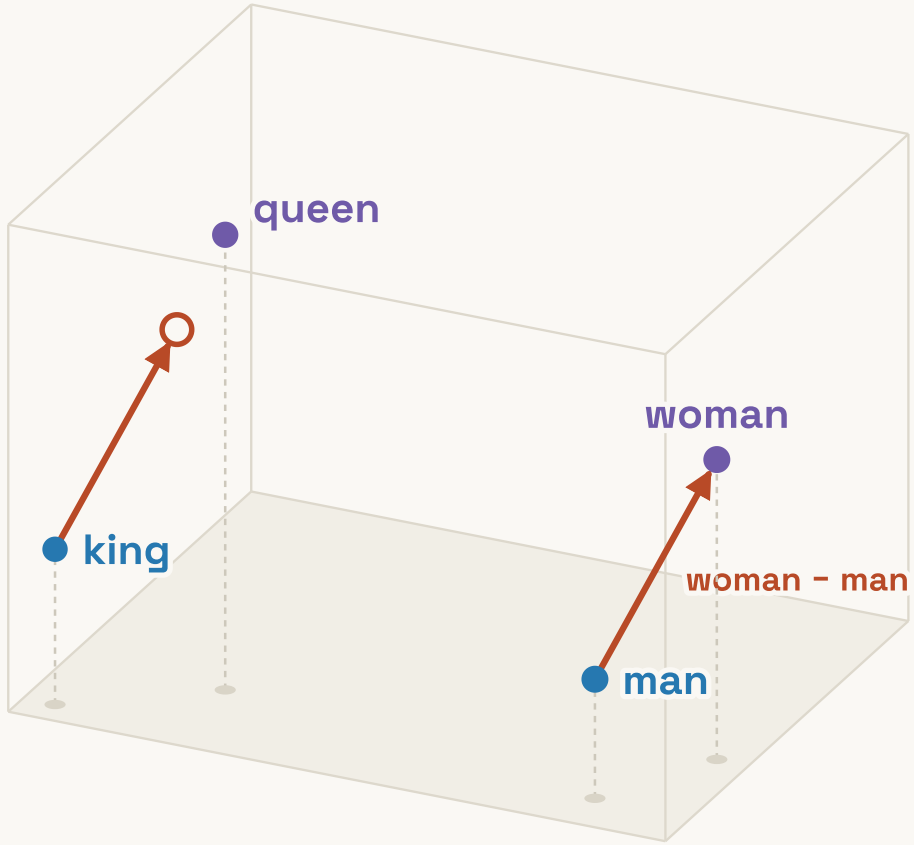
king − man + woman

king

+ woman − man

=

Words as vectors



king − man + woman

king

+ woman − man

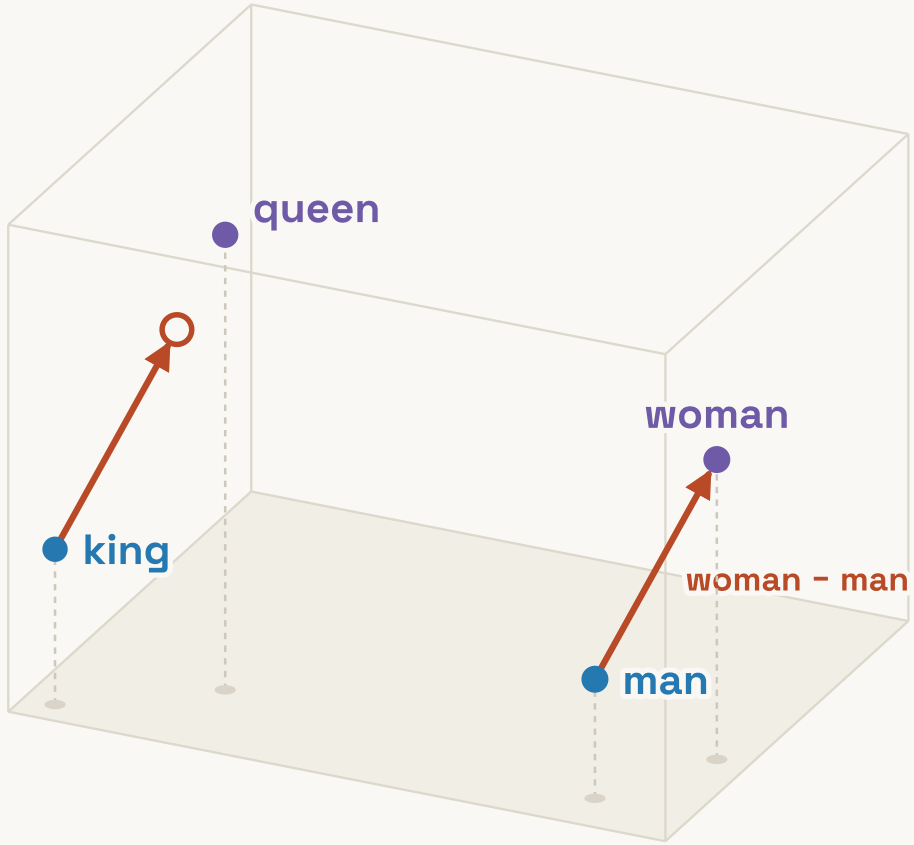
=

queen

the nearest word apart from the three, cosine .86

7 / 8

Words as vectors



king − man + woman

king 

+ woman − man 

=

queen 

the nearest word apart from the three, cosine .86

The drawing projects the six words onto three directions. The sum runs over all fifty numbers.

8 / 8

Where the numbers come from

- The **distributional hypothesis** (Firth, 1957): a word is characterised by the company it keeps
- An **embedding** gives each word *k* numbers, learned from the text alone, so that words used in similar contexts get similar numbers
- **GloVe** (2014) fits them to a corpus's co-occurrence counts; **word2vec** (2013) trains a shallow network to predict a word from its neighbours

```
glove <- read_delim("data/glove.6B.50d.10k.w2v.txt",
                    skip = 1, delim = " ", quote = "",
                    col_names = c("word", paste0("d", 1:50)))

glove |> filter(word %in% c("king", "queen", "pizza")) |>
  select(word, d1:d6)
dim(glove)
```

OUTPUT

```
# A tibble: 3 × 7
  word     d1     d2     d3      d4    d5     d6
  <chr> <dbl>  <dbl>  <dbl>   <dbl> <dbl>  <dbl>
1 king  0.505  0.686 −0.595 −0.0228 0.600 −0.135
2 queen 0.379  1.82  −1.26  −0.104  0.358  0.600
3 pizza 0.621 −0.496 −0.696  0.195  1.06   0.142
```

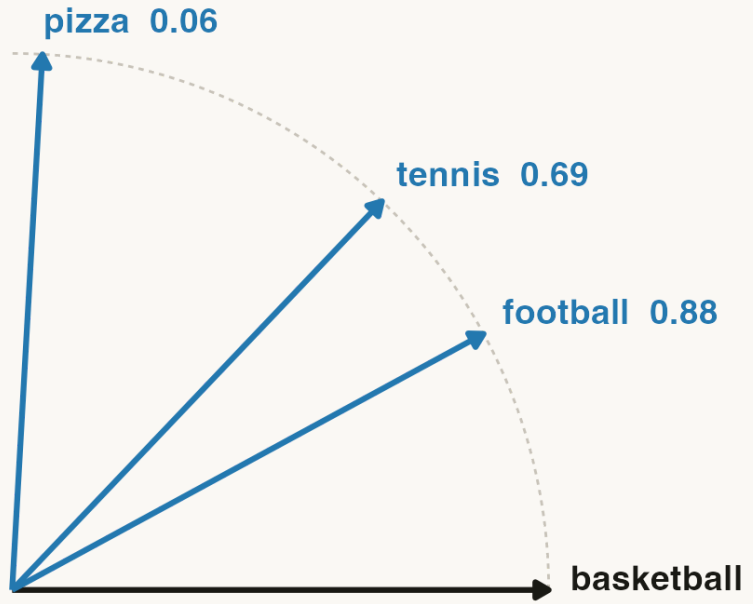
```
[1] 10000    51
```

- Ten thousand words, fifty numbers each, from six billion tokens of Wikipedia and news. No column means anything on its own

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv*. https://doi.org/10.48550/arXiv.1301.3781
Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global vectors for word representation. *Proceedings of EMNLP 2014*, 1532–1543. https://doi.org/10.3115/v1/D14-1162

How close are two words?

- Every word is an arrow. Two words are similar when their arrows point the same way
- **Cosine similarity** measures that angle: 1 for the same direction, 0 for a right angle
- The length of an arrow plays no part, so a frequent word does not win by being long



PLAYGROUND **Multiply, then add**

> *Football* points almost the same way as *basketball*; *pizza* is nearly at a right angle. The neighbour lists on the next slide are sorted by this number.

Neighbours

```
wv <- as.matrix(glove[-1]); rownames(wv) <- glove$word
wv <- wv / sqrt(rowSums(wv^2))     # unit length: dot product = cosine

nearest <- function(target, n = 6) {
  sims <- wv %*% target            # cosine with every word at once
  tibble(word = rownames(wv), similarity = sims[, 1]) |>
    arrange(-similarity) |> slice_head(n = n)
}

nearest(wv["basketball", ])
nearest(wv["netherlands", ])
```

- `nearest()` sorts every word by its cosine with the target
- Nobody told the vectors that Belgium is near the Netherlands. Six billion tokens of context did

OUTPUT

```
# A tibble: 6 × 2
  word       similarity
  <chr>           <dbl>
1 basketball       1
2 football         0.88
3 baseball         0.86
4 hockey           0.86
5 nba              0.84
6 soccer           0.82
```

```
# A tibble: 6 × 2
  word        similarity
  <chr>            <dbl>
1 netherlands       1
2 belgium           0.89
3 switzerland       0.82
4 denmark           0.81
5 france            0.79
6 sweden            0.79
```

PLAYGROUND Where words live

Arithmetic

```r
analogy <- function(a, b, c, n = 4) {
  target <- wv[a, ] - wv[b, ] + wv[c, ]
  nearest(target, n + 3) |>
    filter(!word %in% c(a, b, c)) |>   # the inputs score high: hide them
    slice_head(n = n)
}

analogy("king", "man", "woman")
analogy("doctor", "man", "woman")
```

- The famous demonstration, with its thumb on the scale in view: without the filter, *king* itself is the nearest word
- The second line is the same arithmetic, and the answer is a measurement of the corpus, not of medicine

OUTPUT

```
# A tibble: 4 × 2
  word     similarity
  <chr>         <dbl>
1 queen          0.85
2 throne         0.76
3 prince         0.75
4 daughter       0.74
```

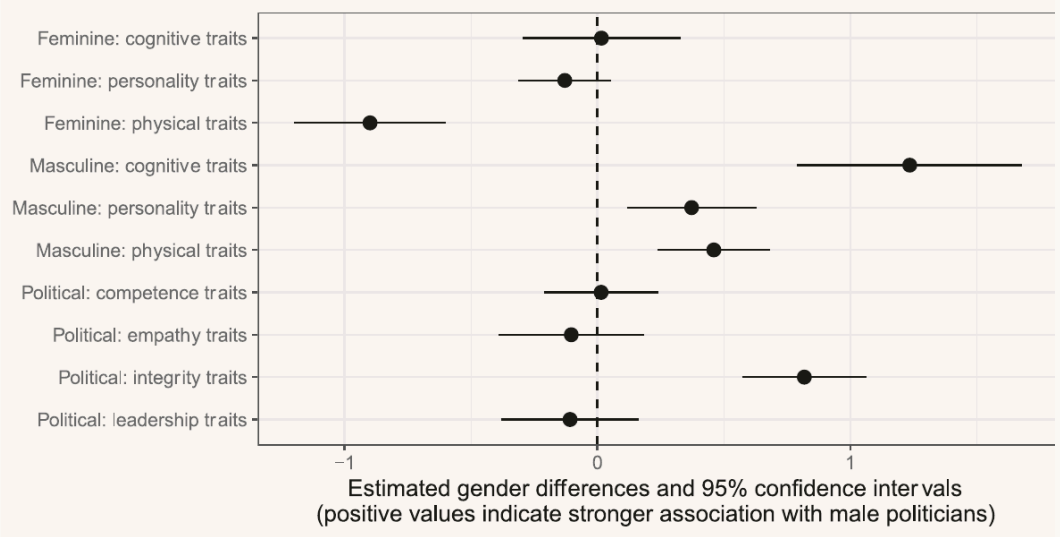
```
# A tibble: 4 × 2
  word     similarity
  <chr>         <dbl>
1 nurse          0.94
2 child          0.86
3 pregnant       0.85
4 mother         0.84
```

> **Embeddings inherit the regularities of the text they were trained on: the useful ones and the ones a study would call bias.**

Bolukbasi, T., Chang, K.-W., Zou, J., Saligrama, V., & Kalai, A. (2016). Man is to computer programmer as woman is to homemaker? Debiasing word embeddings. *Advances in Neural Information Processing Systems, 29*.

Embeddings as the object of study

- Andrich, Bachl and Domahidi (2023): are US politicians described in gender-stereotyped terms in the news?
- Five million news stories, 2010 to 2020; word2vec trained on them, with each of 1,095 politicians as a token of their own
- The measure is a cosine: how close is a politician's vector to words such as *competent*, *honest* or *caring*?



Right of the dashed line: closer to male politicians. Masculine traits and integrity sit closer to men, physical feminine traits closer to women. Most other traits show no difference.

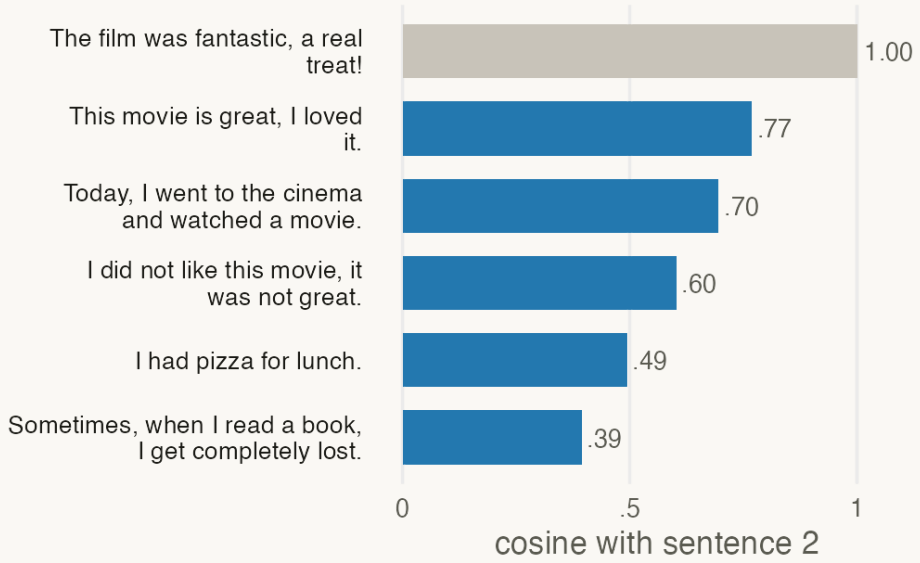
> **No classifier, no labels: the geometry of the space is the finding. The validation question is whether the geometry measures the construct.**

Andrich, A., Bachl, M., & Domahidi, E. (2023). Goodbye, gender stereotypes? Trait attributions to politicians in 11 years of news coverage. *Journalism & Mass Communication Quarterly, 100*(3), 473–497. https://doi.org/10.1177/10776990221142248 Figure 2, cropped (CC BY-NC 4.0).

From words to sentences

```r
movies <- c("This movie is great, I loved it.",
            "The film was fantastic, a real treat!",
            "I did not like this movie, it was not great.",
            "Today, I went to the cinema and watched a movie.",
            "I had pizza for lunch.",
            "Sometimes, when I read a book, I get completely lost.")

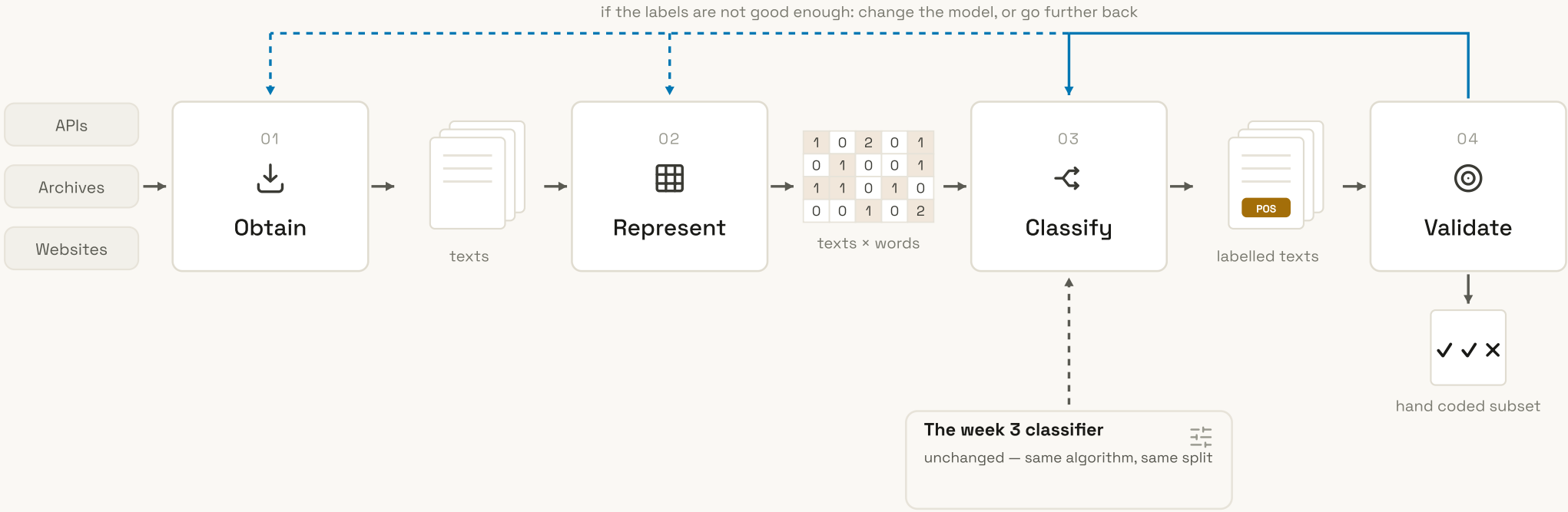
# a sentence-embedding model, run locally through Ollama
emb <- request("http://localhost:11434/api/embed") |>
  req_body_json(list(model = "nomic-embed-text", input = movies)) |>
  req_perform() |> resp_body_json()
E <- do.call(rbind, lapply(emb$embeddings, unlist))   # 6 × 768
E <- E / sqrt(rowSums(E^2))
tibble(sentence = movies, similarity_to_2 = as.numeric(E %*% E[2, ])) |>
  ggplot(aes(similarity_to_2, reorder(sentence, similarity_to_2))) +
  geom_col()
```



- *great* and *fantastic* make the nearest pair: different words, similar meaning. The negated sentence is further away than the cinema trip

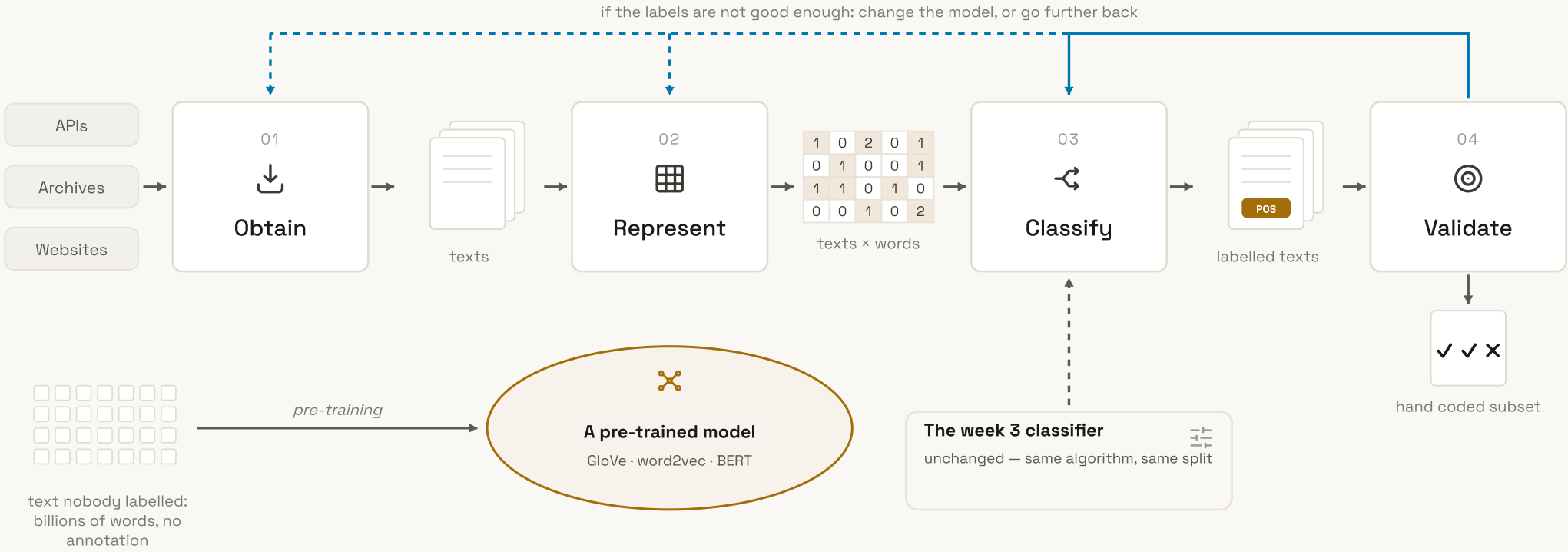
> Averaged word vectors put sentences 1 and 3 almost on the same point. A model that reads the sentence does not.

The pipeline with embeddings

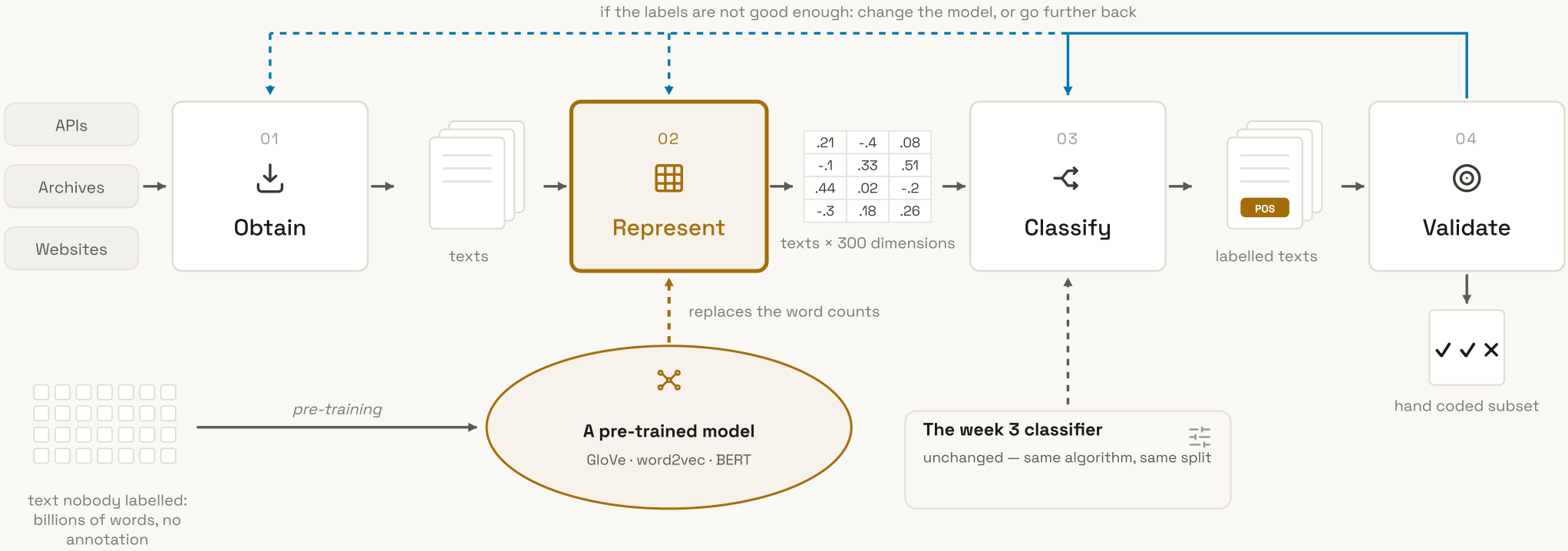


1 / 5

The pipeline with embeddings

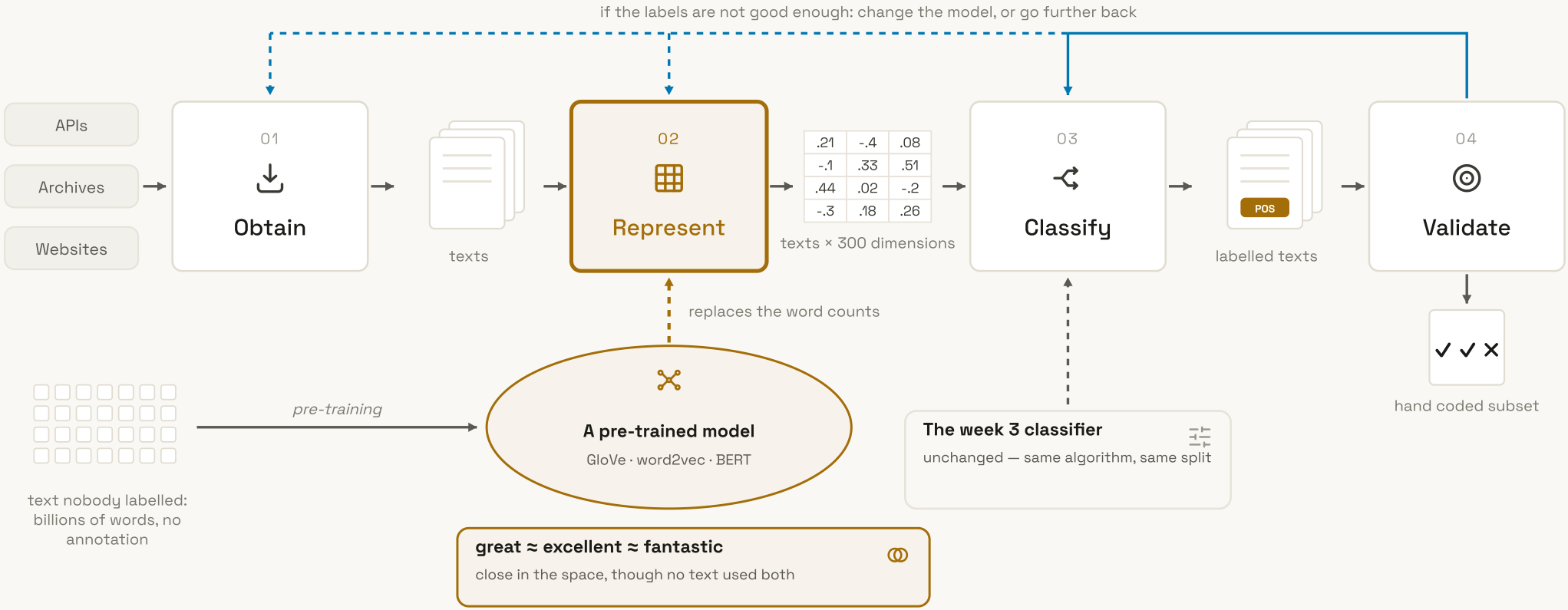


The pipeline with embeddings

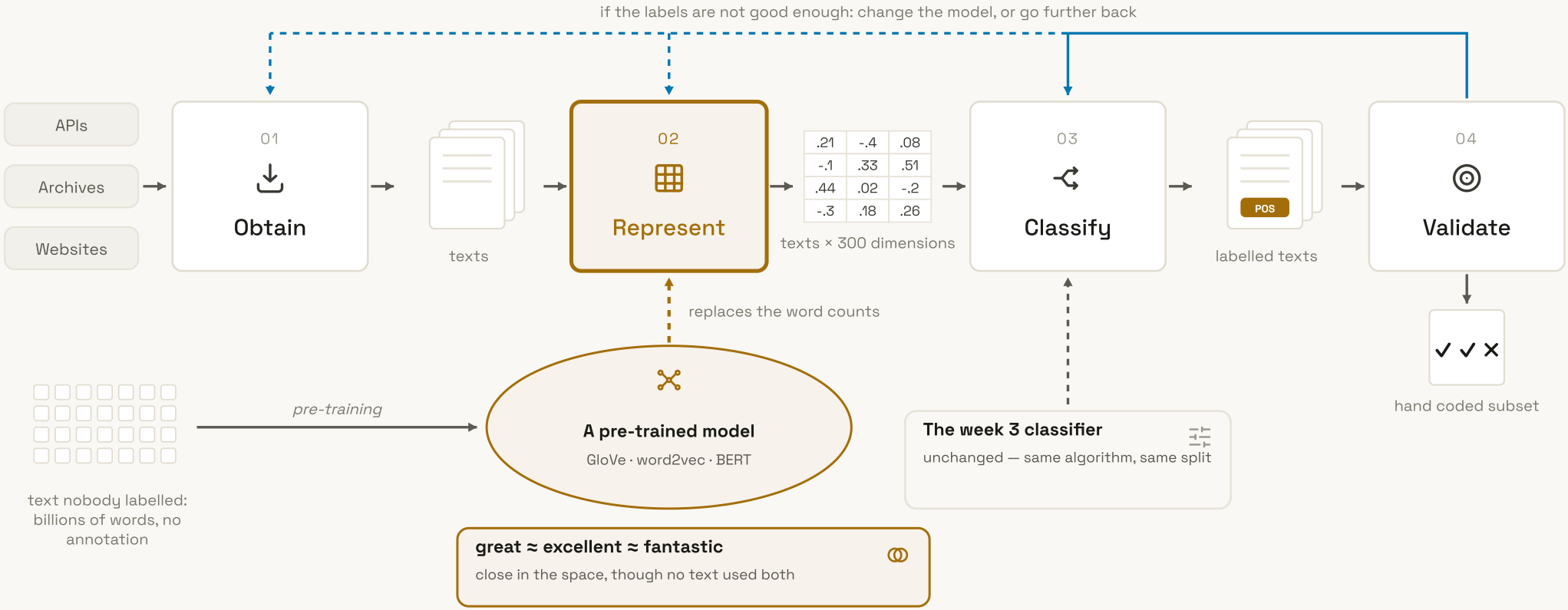


3 / 5

The pipeline with embeddings



The pipeline with embeddings



VU VRIJE UNIVERSITEIT AMSTERDAM

A classifier on embeddings

WORD EMBEDDINGS: FIFTY NUMBERS

```
rec_glove <- recipe(label ~ text, data = train_data) |>
  step_tokenize(text) |>
  step_word_embeddings(text, embeddings = glove,
                       aggregation = "mean")    # mean per abstract

glove_wf <- workflow() |>
  add_recipe(rec_glove) |>
  add_model(logit_spec)                         # Part III's model

m_glove <- fit(glove_wf, data = train_data)

pred_glove <- predict(m_glove, new_data = test_data) |>
  bind_cols(test_data |> select(id, label))
```

SENTENCE EMBEDDINGS: 768 NUMBERS

```
# the request from two slides ago, in batches of 32
emb <- ollama_embed(science$text)               # 5,250 × 768
emb_tbl <- as_tibble(emb,
                     .name_repair = ~ paste0("e", seq_along(.x))) |>
  mutate(id = science$id)

train_emb <- train_data |> select(id, label) |>
  left_join(emb_tbl, by = "id")
test_emb  <- test_data |> select(id, label) |>
  left_join(emb_tbl, by = "id")

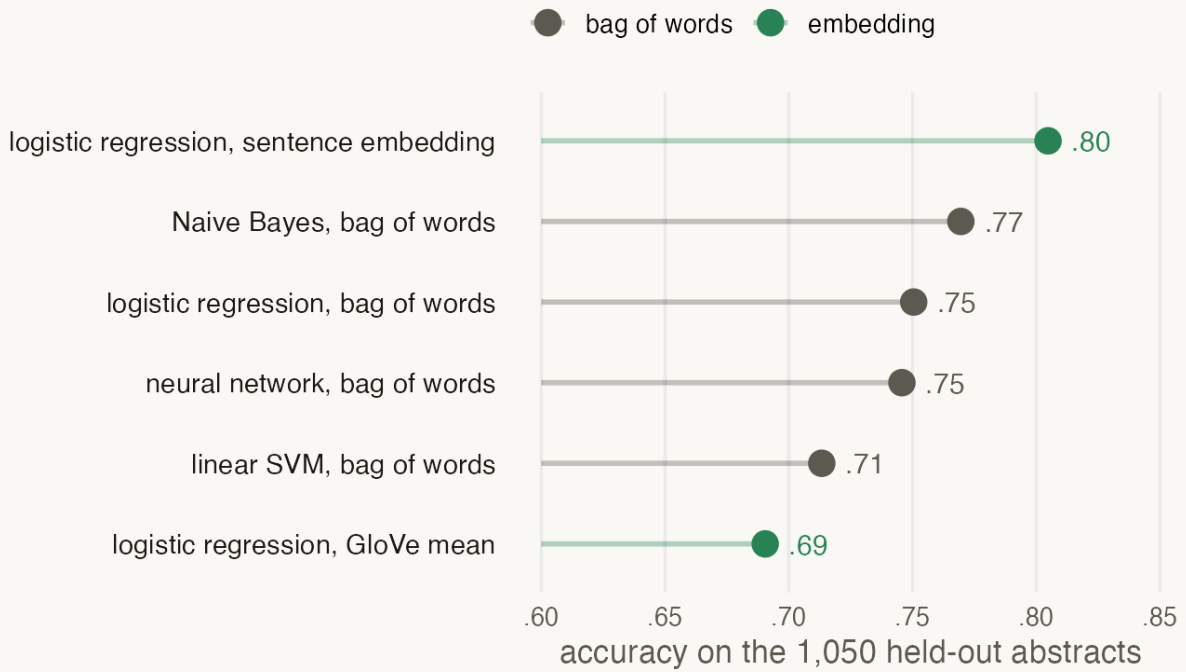
m_sent <- fit(logit_spec, label ~ . - id,       # Part III's model
              data = train_emb)

pred_sent <- predict(m_sent, test_emb) |>
  bind_cols(test_emb |> select(id, label))
```

> The same logistic regression both times. Only what it reads has changed.

Six models, one test set

- The mean of fifty-dimensional word vectors scores below all the bag-of-words models: averaging two hundred words into one point blurs the abstract
- The sentence-embedding model comes first: a representation trained to keep what a text means, fed to the same regression
- The same 1,050 abstracts for all six rows. Every comparison in this course follows that rule



> **Four algorithms on the same word counts span 6 points of accuracy. Changing what one logistic regression reads moves it from .69 to .80.**

VU VRIJE UNIVERSITEIT AMSTERDAM

Tuning: nobody knows the right settings

```
nn_tune <- mlp(hidden_units = tune(),    # "decide this by trying"
               penalty = tune(),
               epochs = tune(),
               learn_rate = tune()) |>
  set_engine("brulee") |> set_mode("classification")

nn_tune_wf <- workflow() |>
  add_recipe(rec_norm) |> add_model(nn_tune)

grid <- crossing(hidden_units = c(2, 4, 8, 16, 32, 64, 128),
                 penalty = c(.0001, .001, .01, .03, .1, .3, 1),
                 epochs = c(5, 100),
                 learn_rate = c(.001, .01, .1))

folds <- vfold_cv(train_data, v = 5, strata = label)   # never the test set

nn_results <- tune_grid(nn_tune_wf, resamples = folds, grid = grid,
                        metrics = metric_set(accuracy))
```

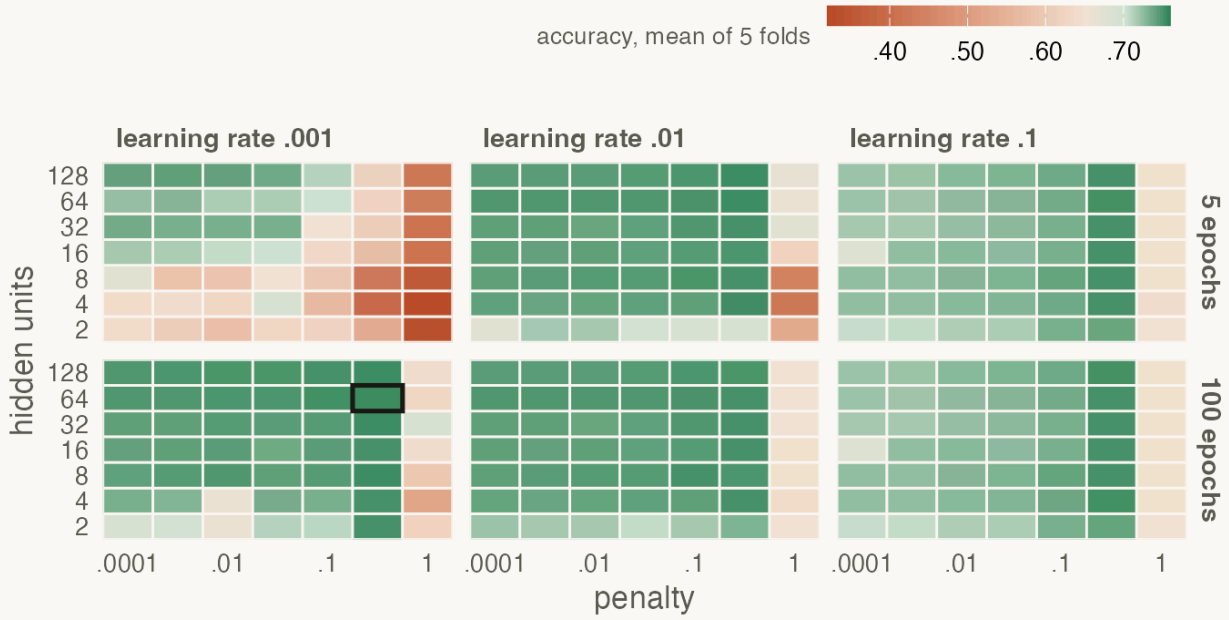
- Hidden units, penalty, epochs, learning rate: every value so far was a guess
- A **grid search** tries the combinations systematically: 294 here, each scored by five-fold cross-validation **inside the training set**
- That makes 1,470 fits, which took ten minutes on twenty processor cores

> **The test set is not touched. Choosing settings on it would be Part II's leak.**

Tuning: what the grid says

```
collect_metrics(nn_results) |>
  ggplot(aes(factor(penalty),
             factor(hidden_units), fill = mean)) +
  geom_tile() +
  facet_grid(epochs ~ learn_rate)
```

- The **penalty** matters most: too little .72, about right .75, too much .65
- Too little training also hurts: at the smallest learning rate, five epochs can stay at chance (.33)
- Hidden units hardly matter, and the top settings all sit near .75



> **Chosen on the training folds, then refitted and scored once on the test set: .77, against .75 for the first guess.**

What moves the score

1. **The task**: sentiment in tweets is harder than discipline in abstracts, whatever the model
2. **The amount of labelled data**: more hand-coding beats a cleverer algorithm, nearly always
3. **The representation**: counts, n-grams, word vectors, sentence vectors; the largest lever today
4. **Preprocessing**: stop words, stemming, trimming; small effects, and not always in the direction you expect
5. **The algorithm**: on a bag of words, a point or two between reasonable choices
6. **Tuning**: a point or two, on folds and not on the test set

> **Items 1 and 2 are decided before any code runs. That is what the sketch at the start of every lab is for.**

Barberá, P., Boydstun, A. E., Linn, S., McMahon, R., & Nagler, J. (2021). Automated text classification of news articles: A practical guide. *Political Analysis, 29*(1), 19–42. https://doi.org/10.1017/pan.2020.8

VI · Weeks 1 to 3 in one table

Where every method lands

APPROACH	ACCURACY	KRIPPENDORFF'S A	WEEK
Manual coding, single coder	.82	.82	2
Manual coding, majority of three	.88	.90	2
Crowd coding, majority of five	.77	.81	2
Machine learning, CNN	**.63**	**.50**	**3**
Machine learning, SVM	**.57**	**.41**	**3**
Machine learning, Naive Bayes	**.58**	**.39**	**3**
Dictionaries, best of fourteen	.46	.34	2
Dictionaries, worst of fourteen	.41	.05	2

Classifiers beat every dictionary and reach no human. Next week adds a row.

van Atteveldt, W., van der Velden, M. A. C. G., & Boukes, M. (2021). The validity of sentiment analysis: Comparing manual annotation, crowd-coding, dictionary approaches, and machine learning algorithms. *Communication Methods and Measures, 15*(2), 121–140. https://doi.org/10.1080/19312458.2020.1869198 — Table 2.

What you can now do

- Say what a classifier needs that a dictionary does not
- Explain overfitting, and why the test set is held back
- Write a workflow: split, recipe, model, workflow, fit, predict, evaluate
- Read a three-class confusion matrix, macro and per class
- Describe a unit, a hidden layer, the loss and backpropagation
- Say what an embedding is and what it changes for a classifier

> **Every score this week was produced the way last week's were: new instruments, the same kind of gold standard.**

This week

- **Tuesday · Lab A**: one workflow, three algorithms, one test set; then try to beat van Atteveldt et al.
- **Thursday · Lab B**: neighbours, analogies, and embeddings instead of word counts for the same classifier
- **Sunday · homework 3**: a classifier for New Year's resolution tweets, on word counts and on embeddings; started in Lab B
- **Playground**: *Let the data write the dictionary* and *How much freedom is too much* before Tuesday; *Where words live* before Thursday
- **Reading**: Barberá et al. (2021); Hvitfeldt & Silge (2021), chapters 6–7; van Atteveldt et al. (2021), the machine learning section

> **Only a score on unseen texts tells you how the model will do. Keep the test set clean.**

VU VRIJE UNIVERSITEIT AMSTERDAM

Questions?

> If anything is unclear, ask now, or on Tuesday with a keyboard in front of you.